

# Optimal Policy Characterization Enhanced Proximal Policy Optimization for Multitask Scheduling in Cloud Computing

Jiangliang Jin<sup>✉</sup> and Yunjian Xu<sup>✉</sup>, *Member, IEEE*

**Abstract**—For a serving system with multiple servers and a public queue, we study the scheduling of multiple tasks with deadlines, under random task arrivals and renewable energy generation. To minimize the weighted sum of the serving cost (associated with the energy consumption) and the delay cost (resulting from deferring the processing of tasks after their deadlines), we formulate the problem as a dynamic program with unknown transition probability. To mitigate the curse of dimensionality, we establish a partial priority rule, the earlier deadline and less demand first (ED-LDF): priority should be given to tasks with earlier deadline and less demand. In the heavy-traffic regime, the established ED-LDF characterization is proved to be optimal under arbitrary system dynamics. We propose a new, scalable ED-LDF-based proximal policy optimization (PPO) approach that integrates our (partial) optimal policy characterizations into the state-of-the-art deep reinforcement learning (DRL) algorithm. Numerical results demonstrate that the proposed ED-LDF-based PPO approach outperforms the classical PPO and three other priority rule-based PPO approaches.

**Index Terms**—Deep reinforcement learning (DRL), delay cost, dynamic programming, mobile-edge computing (MEC), resource allocation.

## I. INTRODUCTION

**W**E STUDY the scheduling of multiple tasks with various demands and deadlines, under state-dependent time-varying processing capacity (in each period), random task arrivals, and serving cost. The scheduling objective is to minimize the long-term expected system cost, which is the weighed sum of the serving costs (e.g., resulting from the energy consumption associated with task processing) and the penalty cost (resulting from delaying the processing of tasks after their deadlines). We consider the stochastic deadline scheduling

problem for a serving system with  $N$  servers and a public queue of  $M$  positions.

One prototype application of the proposed problem is the scheduling of computation tasks in cloud computing [1], where multiple virtual servers (over a physical serving system) are deployed to serve multiple tasks from end users [2]. With the development of the emergent 5G architecture, the mobile-edge computing (MEC) extends cloud computing services to the edge of networks leveraging mobile base stations, where the arriving tasks from smart mobile devices (e.g., smart phones [3], smart vehicles [4], and sensors [5]) are stochastic and impatient. Since the cloud service provider usually has limited amount of virtual resources [5], an important challenge is on the efficient allocation of the limited resources among tasks to achieve lower latency and energy consumption [6]. Similar applications include the packet transmission scheduling in real-time wireless networks [7], [8], and the scheduling of electric vehicles in a charging station [9].

This work is related to three streams of literature on the deadline scheduling of multitask processing, cost minimizing scheduling in cloud computing, and deep reinforcement learning (DRL).

The scheduling of multiple tasks served by multiple servers via a public queue has been extensively studied [1], [2], [10]–[14]. For the queuing of cloud computing systems, fair policies, such as the first-come–first-serve (FCFS) policy and round robin (RR) policy have been widely adopted as a rule of thumb [1], [2], [10], [11]. In the light-traffic case where it is feasible to finish all tasks before their deadlines, both the earliest deadline first (EDF) [12] and the least-laxity first (LLF) [13] policies are optimal for the scheduling of a singer server ( $N = 1$ ). For a multiserver setting without time-varying serving cost or task arrivals, it is shown in [14] that the EDF policy is optimal in the light-traffic case. For the heavy-traffic regime where it is not feasible to finish all tasks before their deadlines, however, it is shown that simple heuristic rules like EDF and LLF may not be optimal [15]. Different from the aforementioned literature, this work focuses on the optimal scheduling of multiple servers in the heavy-traffic regime under random processing costs.

Relatively fewer works have studied the scheduling of multiple servers under random processing costs. Under random processing costs, Yu *et al.* [16] proposed a scalable Whittle Index-based policy approach for the deadline scheduling of multiple tasks. With the incorporation of random renewable

Manuscript received April 27, 2021; revised August 9, 2021; accepted September 4, 2021. Date of publication September 9, 2021; date of current version April 25, 2022. This work was supported in part by the General Research Fund (GRF) Project through the Hong Kong University Grants Committee under Grant 14200720, and in part by the National Natural Science Foundation of China (NSFC) Project under Grant 62073273. (*Corresponding author: Yunjian Xu.*)

Jiangliang Jin is with the Department of Mechanical and Automation Engineering, The Chinese University of Hong Kong, Hong Kong (e-mail: jiangliangjin@cuhk.edu.hk).

Yunjian Xu is with the Department of Mechanical and Automation Engineering, The Chinese University of Hong Kong, Hong Kong, SAR, and also with the Chinese University of Hong Kong Shenzhen Research Institute, Shenzhen 518057, China (e-mail: yjxu@mae.cuhk.edu.hk).

Digital Object Identifier 10.1109/IIOT.2021.3111414

generation, a similar Whittle Index-based policy approach is adopted for the scheduling in MEC [17]. We note, however, that the optimality of Whittle index-based policy is not guaranteed [16], [17]. For IoT systems with random renewable energy, Lee and Lee [18] proposed an asymptotically optimal algorithm where a dynamic priority order is determined by solving a linear programming at each stage. However, the established asymptotically optimality does not necessarily hold under heavy traffic [18]. This work, for the first time, establishes the optimality of a priority rule under heavy traffic and random processing costs.

Optimization-based approaches have been adopted to obtain one-shot solutions for the deadline scheduling of multiple tasks in cloud computing [19]–[23]. Eshraghi and Liang [19] formulated the scheduling problem as a nonconvex mixed-integer problem and propose a nearly optimal algorithm to minimize the weighted sum of the serving cost and delay cost. Chen *et al.* [20] formulated the scheduling problem as a nonconvex separable quadratically constrained quadratic program. To deal with the nonconvexity in the computation resource allocation problem, Qian *et al.* [21] proposed an efficient layered algorithm-based optimization method to find a nearly optimal solution. The scheduling of a green MEC system with the incorporation of renewable energy has been explored through Lyapunov optimization-based and stochastic optimization-based approaches [22], [23].

Receding horizon optimization (RHO)-based approaches have been adopted to obtain online solutions for adaptive cloud computing resource allocation. In [24], a RHO-based online optimization approach is proposed to minimize the maximum communication and computation latency in fog computing under uncertain task arrivals. Since the number of binary variables (indicating whether each unfinished task is delayed or not) increases with the number of tasks, one-shot and RHO-based approaches are, in general, not scalable and cannot deal with large-scale scheduling problems.

DRL methods have demonstrated great potential in dealing with unknown system dynamics and large system state space (e.g., Atari games [25]), through the adoption of nonlinear function approximation schemes (i.e., deep neural networks) which extract key features as a compact representation [26]. Xiong *et al.* [5] formulated the resource allocation problem in MEC with random task arrivals as a Markov decision process (MDP), which is solved by an improved deep Q-network (DQN) algorithm. Huang *et al.* [27] formulated the MEC task scheduling problem with random task arrivals as an infinite-horizon MDP and propose a stochastic gradient descent-based DRL algorithm to solve it. Taking the random renewable generation into consideration, Xu *et al.* [28] proposed an efficient online reinforcement learning-based algorithm for the MEC scheduling.

Although DRL approaches have worked well for small-scale task scheduling problems with a few task arrivals in each period [5], [27], they have difficulties in dealing with large-scale scheduling problems, mainly due to the computational burden and sample efficiency issues brought by the high dimensional action space. For example, in deep Q-learning, the computational burden of finding the greedy action with respect to Q function grows in proportion to the

size of action space [29], [30]. In trust-region-based algorithms, the computation of an approximation to the natural gradient, Fisher-vector products, could be prohibitively expensive for problems with high-dimensional action space [31]. Moreover, the sample size needed to find an  $\epsilon$ -optimal policy is proportional to the size of the action space [32], which grows exponentially with the maximum number of tasks in our context.

The proximal policy optimization (PPO) algorithm is recently proposed to improve the learning speed and to avoid convergence to suboptimal policies for problems with discrete action spaces [33], [34]. This motivates us to combine the established optimal policy characterizations with PPO. Zhan *et al.* [4] proposed a PPO-based offloading scheduling for vehicular edge computing. Different from [4] that focuses on the offloading scheduling of one single smart vehicle, this work studies the cost-minimizing scheduling problem of multiple tasks faced by the service provider (e.g., a cloud computing center).

The main contribution of this work is twofold. First, under a heavy-traffic assumption that the number of unfinished tasks (in the serving system) is no less than the processing capacity (which usually holds when the queue is not empty), we characterize a partial priority order for the preemptive case, the earlier deadline and less demand first (ED-LDF) rule: priority should be given to tasks with earlier deadline and less demand. For an arbitrary policy, we show that it can be improved by the characterized ED-LDF priority order in the heavy-traffic regime, regardless of future system dynamics. To the best of our knowledge, this article is the first that establishes an optimal priority order for resource allocation among multitask queuing system under heavy traffic and random processing costs.

The optimality of the characterized ED-LDF priority order holds under arbitrary trajectories of system uncertainties (in renewable generation, task arrivals, and processing capacity). As a result, it can be integrated into model-free DRL methods that do not require any prior knowledge on the evolution of system uncertainties. The characterized priority rule can reduce the dimensionality of the action space (and the neural network (NN) output) from  $O(M + N)$  to  $O(1)$ . As such, the proposed approach is *scalable* with respect to the maximum number of tasks in the serving system,  $M + N$ .

The second contribution is a systematic approach that combines the advantages of the established optimal priority order and the model-free DRL. The proposed approach first adopts a model-free DRL algorithm (the PPO proposed in [34]) to adaptively learn system dynamics and to determine the total serving amount for the serving system, and then applies the ED-LDF improvement on an existing priority rule to allocate the total serving amount among tasks. As such, the proposed approach does not require any prior knowledge on the evolution of system uncertainties, and can reduce the dimensionality of NN outputs (through the incorporation of optimal policy characterizations).

We demonstrate the effectiveness of the proposed approach by numerically comparing it with the classical PPO method and three priority rule-based PPO approaches, PPO+EDF

(earliest deadline first), PPO+LLF (least laxity first) and PPO+RR (round robin). Numerical results demonstrate that the proposed approach significantly improves the learning speed of the classical PPO methods in the training phase, and outperforms various benchmark solutions. Compared to PPO+EDF, PPO+LLF, PPO+RR, and classical PPO, the proposed approach reduces 1%–5%, 25%–35%, 53%–80%, and 94%–98% of the average total cost in preemptive scheduling, respectively, and reduces 1%–5%, 36%–54%, 46%–77%, and 93%–98% of the average total cost in nonpreemptive scheduling, respectively.

The remainder of this article is organized as follows. In Section II, we formulate the multitask scheduling problem as a dynamic program with unknown state transition probability. In Section III, we establish the optimality of a partial priority order (ED-LDF) under heavy traffic assumptions. In Section IV, we propose a novel approach that integrates the optimal policy characterization established in Section III into the PPO algorithm. In Section V, we present numerical results to validate the effectiveness of the proposed approach under different parameter settings.

## II. SYSTEM MODEL AND PROBLEM FORMULATION

We study the multitask scheduling problem in a serving system with  $N$  servers and a public queue with the maximum length of  $M$ . This sequential decision-making problem has the following elements.

### A. System Model

- 1) *Discrete Time*: Let  $\mathcal{T} = \{0, \dots, T\}$  denote the set of discrete operating stages.
- 2) *Serving System*: We consider a serving system with  $N$  identical servers deployed at the same place with a public queue of  $M$  positions. We assume that all servers have the same serving rate as in [35]. Let  $\mathcal{N} = \{1, \dots, N\}$  denote the set of servers and  $\mathcal{M}$  be the set of positions of the public queue. Let  $\mathcal{I}_t \subset \{1, \dots, M+N\}$  denote the set of tasks in the serving system at stage  $t$ .
- 3) *Randomness*: For each stage  $t \in \mathcal{T}$ , let  $s_t \in \mathcal{S}$  denote the global state, where  $\mathcal{S}$  is a finite set. The (possibly multidimensional) global state  $s_t$  contains all information about the exogenous factors that may impact future task arrivals and renewable energy generation, such as the number of the mobile devices in the service region, existing computation demand of mobile devices, and the weather condition. The global state  $s_t$  evolves as an exogenous time-homogeneous Markov chain, and its (unknown) state transition is assumed to be independent of the taken actions.
- 4) *Task Arrival*: At the beginning of each stage  $t$ , the number of newly arrived tasks and their initial states are determined by the global state  $s_t$ . Once a task arrives at the cloud computing platform via wireless communication, it reports its demand  $d_{i,t}$  and deadline  $t_i^d$  to the service provider [3], where  $d_{i,t}$  is the number of computation time units (a nonnegative integer) needed to complete task  $i$ .

- 5) *Renewable Energy*: For  $t \in \mathcal{T}$ , let  $g_t(s_t)$  denote the renewable energy captured at stage  $t$ , which can be shared by different servers in the serving system. The renewable energy (e.g., solar generation) has been widely adopted to reduce the energy cost of cloud computing at data centers [1], [36].
- 6) *Discrete Serving Action*: For each task  $i \in \mathcal{I}_t$  in the serving system, we use

$$a_{i,t} \in \{0, 1\} \quad (1)$$

to denote the serving decision: 1 for serving and 0 for not serving. At each stage, the serving action is an  $(N+M)$ -dimensional vector  $\mathbf{a}_t = (a_{1,t}, \dots, a_{N+M,t})$ .

- 7) *Delayed Tasks*: Each task  $i$  leaves the serving system at the end of its deadline  $t_i^d$  if it is fully served. Otherwise, the delayed task remains in the serving system until being fully served. Let  $\tau_{i,t} = t_i^d - t + 1 \leq \tau^{\max}$  denote task  $i$ 's remaining serving stages at stage  $t$ , where  $\tau^{\max}$  is an integer upper bound on the number of stages a task can stay in the system. We use

$$\mathcal{D}_t = \{i \mid d_{i,t} > 0, \tau_{i,t} \leq 0, i \in \mathcal{I}_t\} \quad (2)$$

to denote the set of delayed tasks at stage  $t$ .

*Remark 1*: It is worth noting that the reported deadline  $t_i^d$  usually refers to the deadline when the customer must receive the finished computing task. As a result, the service provider needs to deduct the communication delay from the remaining processing time, as communication latency could constitute a nonnegligible part of the total offloading delay [35].

To calculate the communication delay, the service provider can first estimate/calculate the communication rate, as a function of the bandwidth of the channel, the transmission power, the channel gain of the base station, and the density of noise power of the channel [3], [35]. Then, the service provider can approximate the communication delay as the value of task size divided by the communication rate [3], [35]. When the communication delay is relatively large (e.g., due to the low communication rate), the aforementioned procedure is needed to accurately calculate the processing deadline of a task.

### B. Markov Decision Process

We formulate the multitask scheduling problem as a  $(T+1)$ -stage MDP (with unknown state transition probability) by introducing its system state, action space, transition process, and stage cost.

Let  $x_{i,t} = (d_{i,t}, \tau_{i,t})$  denote the state of task  $i$ , where  $d_{i,t}$  and  $\tau_{i,t}$  denote its remaining demand and remaining serving periods, respectively. At stage  $t$ , the laxity of task  $i$  is defined as

$$\theta_{i,t} = \tau_{i,t} - d_{i,t} \quad \forall i \in \mathcal{I}_t$$

which is the difference between the number of remaining serving stages and the remaining demand. An illustration of task  $i$ 's state is given in Fig. 1. For  $j \in \{1, \dots, M+N\} \setminus \mathcal{I}_t$ , we let  $x_{j,t} = (0, 0)$  for notational convenience.

The serving amount of task  $i$  cannot exceed its remaining demand, i.e.,

$$0 \leq a_{i,t} \leq d_{i,t} \quad \forall i \in \mathcal{I}_t. \quad (3)$$

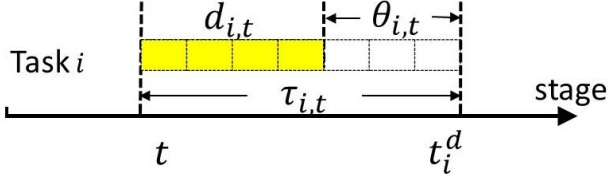


Fig. 1. Illustration for the state of task  $i$  at stage  $t$ , with remaining demand  $d_{i,t} = 4$  and remaining serving stages  $\tau_{i,t} = 7$ , and laxity  $\theta_{i,t} = \tau_{i,t} - d_{i,t} = 3$ .

We use vector  $\ell_t$  to record the indices of tasks connected to the servers at the end of stage  $t$ , i.e.,  $\ell_t = (\ell_{1,t}, \dots, \ell_{N,t})$  where  $\ell_{n,t}$  is the index of the task at server  $n$  at stage  $t$ . For an empty server  $n$ , we let  $\ell_{n,t} = 0$ . Since one task cannot be served by two servers at the same time, for any pair of servers  $m, n \in \mathcal{N}$ , we have

$$\ell_{n,t} \neq \ell_{m,t} \quad \forall m, n \in \mathcal{N}. \quad (4)$$

The system state (of the formulated MDP) is given by  $(\mathbf{x}_t, s_t)$ , where the endogenous component

$$\mathbf{x}_t = (x_{1,t}, \dots, x_{M+N,t}, \ell_{t-1})$$

represents the status of the tasks in the serving system at the beginning of stage  $t$ .<sup>1</sup> The global state  $s_t$  governs the evolution of the exogenous system uncertainties.

For task  $i$  with remaining demand  $d_{i,t} \geq 1$ , or with  $d_{i,t} = 0$ ,  $\tau_{i,t} > 1$ , its state evolves as

$$x_{i,t+1} = x_{i,t} - (a_{i,t}, 1). \quad (5)$$

For task  $i$  with  $\tau_{i,t} \leq 1$  and  $d_{i,t} - a_{i,t} = 0$ , it leaves the serving system at the end of stage  $t$ . At the beginning of stage  $t+1$  (before the possible task arrivals), let  $\mathcal{J}_{t+1}$  denote the set of tasks that remain in the serving system, i.e.,

$$\mathcal{J}_{t+1} = \mathcal{D}_{t+1} \cup \{i \mid \tau_{i,t+1} \geq 1, i \in \mathcal{I}_t\} \quad (6)$$

where the set of delayed tasks is defined in (2). Note that  $\mathcal{J}_{t+1} \subset \mathcal{I}_t$ . An empty index  $i \notin \mathcal{J}_{t+1}$  can be used to index a new task that arrives at the beginning of stage  $t+1$ , with  $i \in \mathcal{I}_{t+1}$  and  $x_{i,t+1}$  denoting the initial state of this task.

The system state  $(\mathbf{x}_t, s_t)$  contains an *endogenous* part  $\mathbf{x}_t \in \mathcal{X}$  whose deterministic evolution is governed by the taken action, and an *exogenous* component  $s_t \in \mathcal{S}$  whose stochastic evolution is governed by nature. This allows us to consider a state transition probability  $P: \mathcal{X} \times \mathcal{S} \times \mathcal{A} \times \mathcal{X} \times \mathcal{S} \rightarrow [0, 1]$  to describe the system dynamics, where  $P(\mathbf{x}_{t+1}, s_{t+1} \mid \mathbf{x}_t, s_t, \mathbf{a}_t)$  denotes the probability of state  $(\mathbf{x}_{t+1}, s_{t+1})$  at stage  $t+1$  given the state-action pair  $(\mathbf{x}_t, s_t, \mathbf{a}_t)$  at stage  $t$ . Since the state transition of the exogenous component  $s_t$  is usually unknown in practice, the proposed framework does not require any prior information on the evolution of  $s_t$ , which is adaptively learned by model-free DRL approaches.

Let  $u_t$  denote the total amount of service provided at stage  $t$ , which is constrained by the state-dependent maximum serving

capacity of the system, i.e.,

$$0 \leq u_t = \sum_{i \in \mathcal{I}_t} a_{i,t} \leq C_t(s_t) \leq N \quad \forall t \in \mathcal{T} \quad (7)$$

where  $C_t(s_t)$  is the maximum serving capacity determined by the global state.

As in [1] and [37], we assume that the energy consumption of the serving system,  $F(u_t)$ , is nondecreasing and convex in  $u_t$ . For each  $t < T$ , the *stage cost* is the weighted sum of the serving cost and the penalty cost (resulting from service delays)

$$w_t(\mathbf{x}_t, s_t, \mathbf{a}_t) = \beta \max(0, F(u_t) - g_t(s_t)) + (1 - \beta)|\mathcal{D}_t| \quad (8)$$

where  $F(u_t)$  is convex in  $u_t$ ,  $|\mathcal{D}_t|$  is the cardinality of  $\mathcal{D}_t$ , and  $\beta \in [0, 1]$ . We assume free disposal of renewable energy in the first term in (8) as in [38] and [39].<sup>2</sup> According to (8), the service provider first uses renewable generation to meet the energy demand; when the renewable energy is not sufficient, energy procured from the grid will be used to power the servers. When  $\beta = 0$  and  $\gamma = 1$ , the objective of the proposed MDP is to minimize  $\sum_t |\mathcal{D}_t|$ , which is equivalent to minimizing the total processing time of all delayed tasks (a common setting in the cloud computing literature [21], [40]).

Let  $t_{\max}^d < T$  denote the latest deadline of all tasks. Without loss of generality, the final stage  $T$  is set to guarantee that all delayed tasks can be finished before  $T$ . To enforce the completion of all tasks, we set the terminal cost as follows:

$$w_T(\mathbf{x}_T, s_T) = \begin{cases} 0, & \text{if } \mathbf{x}_T = ((0, 0), \dots, (0, 0), \ell_{T-1}) \\ \infty, & \text{otherwise.} \end{cases} \quad (9a) \quad (9b)$$

For the scheduling problem with  $N$  servers and a public queue, we consider the following two cases of practical importance.

1) *Preemptive Scheduling*: The preemptive scheduling, where the service of a task can be interrupted by another task with higher priority at any time, has been adopted to model the computation resources allocation for offloading tasks at MEC servers [3], [40]. In the preemptive scheduling, the matching policy  $\pi_t^P$  maps the system state  $(\mathbf{x}_t, s_t) \in \mathcal{X} \times \mathcal{S}$  to the matching vector  $\ell_t$ , without any restriction from the previous matching vector, i.e.,

$$\pi_t^P: \mathcal{X} \times \mathcal{S} \rightarrow \{\ell_t \mid \ell_t \in \mathbf{R}^N, (4)\}. \quad (10)$$

We use  $\pi^P = (\pi_0^P, \dots, \pi_{T-1}^P)$  to denote the matching policy in the preemptive case.

2) *Nonpreemptive Scheduling*: In the nonpreemptive case, a server is occupied by its task until the task leaves the serving system [19]. We therefore have

$$\ell_{n,t} = \ell_{n,t-1} \quad \forall \ell_{n,t} \in \mathcal{J}_t \quad \forall n \in \mathcal{N}. \quad (11)$$

The matching policy  $\pi_t^{NP}: (\mathbf{x}_t, s_t) \rightarrow \ell_t$  in the nonpreemptive case needs to respect the constraint in (11), i.e.,

$$\pi_t^{NP}: \mathcal{X} \times \mathcal{S} \rightarrow \{\ell_t \mid \ell_t \in \mathbf{R}^N, (4), (11)\}. \quad (12)$$

<sup>2</sup>In real-world applications, the disposal of renewable energy can be achieved by disconnecting the renewable generators from the system at neglectable costs. We note that the assumption of free renewable disposal has been widely (explicitly or implicitly) adopted in the literature when calculating the renewable curtailment cost [38], [39].

<sup>1</sup>Note that for the preemptive case, the system state does not need to include  $\ell_{t-1}$ .

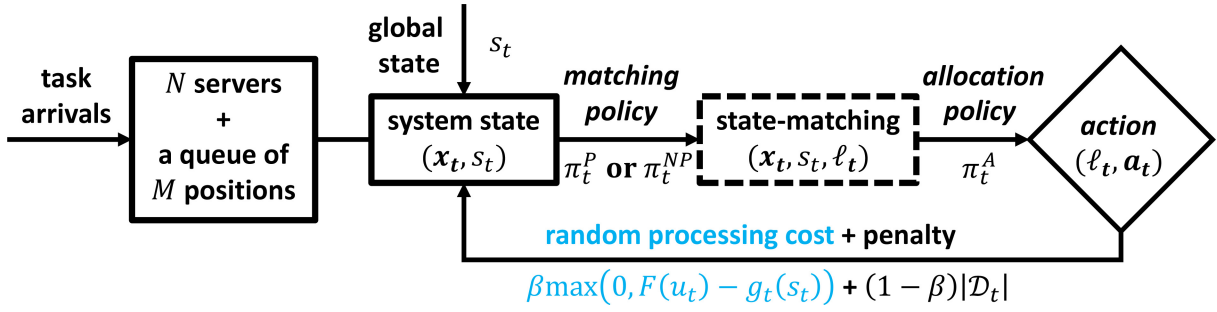


Fig. 2. Illustration of the system model.

Let

$$\mathcal{L}_t \triangleq \{i \mid \ell_{n,t} = i \quad \forall n = 1, \dots, N, i \in \mathcal{I}_t\} \quad (13)$$

denote the set of tasks connected to the  $N$  servers in period  $t$ .

*Remark 2:* For the preemptive case, since the service of a task can be interrupted without incurring any cost [cf. (10)], given the total service amount  $u_t$ , a (partial) priority rule that is optimal for the allocation policy  $\pi^A$  must also apply to the matching policy  $\pi^P$ . We note that this is not true for the nonpreemptive case, as the matching of a task to a server incurs significant opportunity cost.

All tasks in the queue cannot be served, i.e.,

$$a_{i,t} = 0 \quad \forall i \notin \mathcal{L}_t. \quad (14)$$

Given the current system state  $(\mathbf{x}_t, s_t)$ , a feasible  $(N + M)$ -dimensional serving action  $\mathbf{a}_t$  lies in the action space  $\mathcal{A}(\mathbf{x}_t, s_t)$ , where

$$\mathcal{A}(\mathbf{x}_t, s_t) \triangleq \{\mathbf{a}_t \mid (1), (3), (7), (14)\}. \quad (15)$$

We use  $\pi^A = (\pi_0^A, \dots, \pi_{T-1}^A)$  to denote the allocation policy, with  $\pi_t^A : (\mathbf{x}_t, s_t, \ell_t) \rightarrow \mathbf{a}_t \in \mathcal{A}(\mathbf{x}_t, s_t)$ . We note that  $(\pi^P, \pi^A)$  and  $(\pi^{NP}, \pi^A)$  constitute a complete policy  $\pi = (\pi_0, \dots, \pi_T)$ , for the preemptive and nonpreemptive cases, respectively.

We use  $J_t^\pi(\mathbf{x}_t, s_t)$  to denote the cost-to-go function under a policy  $\pi$  and the system state  $(\mathbf{x}_t, s_t)$

$$J_t^\pi(\mathbf{x}_t, s_t) = w_t(\mathbf{x}_t, s_t, \pi_t(\mathbf{x}_t, s_t)) + \mathbb{E} \left\{ \sum_{\tau=t+1}^T \gamma^{\tau-t} w_\tau(\mathbf{x}_\tau, s_\tau, \pi_\tau(\mathbf{x}_\tau, s_\tau)) \right\} \quad (16)$$

where  $\gamma \in (0, 1]$  is the discount factor, and the expectation is over the sequence of global states  $\{s_\tau\}_{\tau=t+1}^T$ . Since the set of global states  $\mathcal{S}$  is finite, according to (9a) and (9b) the cost-to-go function is bounded under the policies which complete all tasks before the final stage  $T$ . We use  $\Pi$  to denote the set of these policies

$$\Pi = \{\pi \mid J_t^\pi(\mathbf{x}_t, s_t) < \infty \quad \forall (\mathbf{x}_t, s_t)\}. \quad (17)$$

Since we have assumed there are enough stages from the latest deadline (of all tasks)  $t_{\max}^d$  to  $T - 1$  to finish all tasks before  $T$ , the set  $\Pi$  is not empty.

By a slight abuse of notation, we use  $J_t(\mathbf{x}_t, s_t)$  to denote the optimal cost-to-go function, i.e.,

$$J_t(\mathbf{x}_t, s_t) \triangleq \inf_{\pi \in \Pi} \{J_t^\pi(\mathbf{x}_t, s_t)\}. \quad (18)$$

We say a policy  $\pi^*$  is optimal, if it attains the optimal cost-to-go defined above, i.e.,  $J_0^{\pi^*}(\mathbf{x}_0, s_0) = J_0(\mathbf{x}_0, s_0)$ , for all initial states  $(\mathbf{x}_0, s_0)$ .

*Remark 3:* It is worth noting that the formulated MDP cannot be solved by brute-force dynamic programming approaches, due to the *curse of dimensionality* in both the system state and action spaces (which grow exponentially with the maximum number of tasks in the system,  $N + M$ ). The evolution of the system uncertainties, e.g., in task arrival, renewable energy, and maximum serving capacity, is usually not perfectly known, which further complicates the scheduling problem.

An illustration of the aforementioned system model is given in Fig. 2.

### III. EARLIER DEADLINE AND LESS DEMAND FIRST PRIORITY RULE

Existing (deep) reinforcement learning methods cannot efficiently deal with the formulated large-scale scheduling problem, due to the high dimensionality of the action space. In particular, the number of possible serving actions  $\mathbf{a}_t$  can be as large as  $2^{N+M}$ , where  $N + M$  is the maximum number of tasks in the serving system. We are therefore motivated to consider a new, scalable approach that integrates the structural characterizations of optimal scheduling policies into DRL, so as to reduce the action space dimensionality without loss of generality.

According to (7), the feasible space of the total amount of service provided at stage  $t$ ,  $u_t$ , grows linearly with the number of servers  $N$ . We propose a new approach that first adopts DRL to decide  $u_t$  and then applies the characterized priority rules to determine the matching and serving vectors  $(\ell_t, \mathbf{a}_t)$  based on  $u_t$ . As such, the proposed approach significantly reduces the dimensionality of DRL outputs (from an  $N + M$  dimensional vector  $\mathbf{a}_t$  to a scalar  $u_t$ ).

To explore the optimal priority rule (among all unfinished tasks in the serving system) we first study the priority between two tasks with the same remaining demand and different deadlines.

*Lemma 1:* For both preemptive and nonpreemptive scheduling, given a system state  $(\mathbf{x}_t, s_t)$  and two tasks connected to the servers  $i, j \in \mathcal{L}_t$ , with  $d_{i,t} = d_{j,t}$  and  $\tau_{i,t} \geq \tau_{j,t}$ , for any policy  $\pi = (\pi^P, \pi^A)$  or  $(\pi^{NP}, \pi^A)$  that serves task  $i$  but not  $j$  at  $(\mathbf{x}_t, s_t)$ , there exists a policy  $\bar{\pi} = (\bar{\pi}^P, \bar{\pi}^A)$  or  $(\bar{\pi}^{NP}, \bar{\pi}^A)$  that serves task  $j$  but not  $i$  at  $(\mathbf{x}_t, s_t)$ , such that

$$J_t^{\bar{\pi}}(\mathbf{x}_t, s_t) \leq J_t^{\pi}(\mathbf{x}_t, s_t). \quad (19)$$

The proof of Lemma 1 is given in Appendix A. Lemma 1 establishes a partial priority order for the allocation policy  $\pi^A$ : for tasks with the same demand, the ones with earlier deadline should be served first.

The following lemma introduces an optimal partial order for the matching policy.

*Lemma 2:* For both preemptive or nonpreemptive scheduling, given a system state  $(\mathbf{x}_t, s_t)$  and two tasks connected to the servers  $i, j \in \mathcal{L}_t$ , with  $d_{i,t} = d_{j,t}$  and  $\tau_{i,t} \geq \tau_{j,t}$ , for any policy  $\pi = (\pi^P, \pi^A)$  or  $(\pi^{NP}, \pi^A)$  that matches task  $i$  but not  $j$  to the server, there exists a policy  $\bar{\pi} = (\bar{\pi}^P, \bar{\pi}^A)$  or  $(\bar{\pi}^{NP}, \bar{\pi}^A)$  that matches task  $j$  but not  $i$  to the server, such that (19) holds.

For the case with preemptive scheduling, Lemma 2 directly follows from Lemma 1 (cf. the discussion in Remark 2). Hence we only need to prove Lemma 2 for the nonpreemptive scheduling case in Appendix B.

The usefulness of Lemma 2 is to improve an existing priority rule (e.g., FCFS) by reordering tasks with the same remaining demand when determining  $\ell_t$ .

Since the delay cost is in proportion to the number of delayed tasks, for tasks with the same deadline, it is natural to serve tasks with less remaining demand first to reduce the number of delayed tasks. However, as shown in the following example, since the less remaining demand first rule minimizes the number of unfinished tasks, it may reduce the size of future action spaces, leading to higher (energy and delay) cost in the future.

*Example 1:* Consider two tasks with states  $x_{1,0} = (1, 2)$ ,  $x_{2,0} = (2, 2)$ . Consider a simple case with the energy cost weight  $\beta = 0$ , the discount factor  $\gamma = 1$ ,  $C_t(s_t) = 1$  for  $t = 0, 2$  and  $C_1(s_1) = 2$ .

If we give priorities to tasks with less remaining demand, the actions are  $\mathbf{a}_0 = (1, 0)$ ,  $\mathbf{a}_1 = (0, 1)$ , and  $\mathbf{a}_2 = (0, 1)$ . The less remaining demand first rule completes task 1 at stage 0, shrinks the action space at stage 1, and results in 1 unit of delay cost. Giving priorities to tasks with more remaining demand, the service provider will take the actions  $\mathbf{a}_0 = (0, 1)$ ,  $\mathbf{a}_1 = (1, 1)$ , and  $\mathbf{a}_2 = (0, 0)$ , leading to zero delay cost. In this example, the less demanding first priority rule performs worse than its opposite due to the action space reduction at stage 1.

To formally prove the optimality of the less remaining demand first priority rule in the heavy traffic regime, we need the following definition.

*Definition 1 (Heavy Traffic):* The serving system is under heavy traffic at stage  $t$ , if any feasible policy  $\pi \in \Pi$  must serve tasks at stage  $t$  to ensure the completion of all tasks before stage  $T$ , i.e.,

$$u_t > 0 \quad \forall \pi \in \Pi \quad (20)$$

where  $\Pi$  is the set of policies which can complete all tasks before stage  $T$ , and the number of unfinished tasks in the system exceeds the maximum serving capacity, i.e.,

$$|\{i \mid d_{i,t} \geq 1, i \in \mathcal{I}_t\}| > C_t(s_t) \quad (21)$$

where  $|\cdot|$  denotes the cardinality of the set.

*Remark 4:* Equation (20) requires that any policy  $\pi \in \Pi$  that can finish all tasks before deadline  $T$  must serve at least one task at stage  $t$ . Equation (21) usually holds when the queue is not empty. The heavy-traffic condition defined by (20) and (21) has been observed in cloud computing [10] and edge computing [41] when the workload is heavy with respect to the limited computing resource.

At system state  $(\mathbf{x}_t, s_t)$ , let  $t_i^{\max}(\mathbf{x}_t, s_t)$  denote the latest departure time of task  $i$  under any feasible policy in the set  $\Pi$  and global state trajectory  $\{s_k\}_{k=t}^{T-1}$ , i.e.,

$$t_i^{\max}(\mathbf{x}_t, s_t) = \max_{\pi \in \Pi} \max_{\{s_k\}_{k=t}^{T-1}} \{k \mid i \in \mathcal{I}_k, i \notin \mathcal{J}_{k+1}\} \quad (22)$$

where  $\mathcal{I}_k$  is the set of tasks in the serving system at stage  $k$ ,  $\mathcal{J}_{k+1}$  is the set of tasks in the serving system at the beginning of stage  $k+1$  [cf. (6)]. Note that both  $\mathcal{I}_k$  and  $\mathcal{J}_{k+1}$  are determined by the policy and the global state trajectory.

*Assumption 1 (Heavy-Traffic Assumption):* Given the system state  $(\mathbf{x}_t, s_t)$ , the heavy-traffic assumption holds for tasks  $i$  if  $\theta_{i,t} \leq 0$ , and the serving system is under heavy-traffic from stage  $t$  to  $t_i^{\max}(\mathbf{x}_t, s_t)$ , in the sense of Definition 1.

The condition  $\theta_{i,t} \leq 0$  implies that task  $i$  will incur more delay cost if it is not served at stage  $t$ . We note that this condition usually holds for latency-sensitive tasks in MEC [4], [40]. Note that under Assumption 1, it is not feasible to finish all tasks before their deadlines (in all possible state trajectories in the future). As a result, simple heuristic rules, such as EDF and LLF may not be optimal [15].

*Definition 2 (Earlier Deadline and Less Remaining Demand First (ED-LDF) Order):* At stage  $t$ , for two tasks  $i$  and  $j$  in the set  $\mathcal{I}_t$ , we say  $i \leq j$  under the ED-LDF order if and only if  $d_{i,t} \geq d_{j,t}$ , and  $t_i^d \geq t_j^d$ . We say task  $j$  is prior to task  $i$  under the ED-LDF order ( $i < j$ ), if and only if  $i \leq j$  and  $x_{i,t} \neq x_{j,t}$ .

Since the ED-LDF relation  $\leq$  defined in Definition 2 is reflexive, antisymmetric, and transitive, it is a partial order. The following theorem ensures the optimality of ED-LDF order under mild assumptions.

*Theorem 1:* In preemptive scheduling, given a system state  $(\mathbf{x}_t, s_t)$  with tasks  $i, j \in \mathcal{I}_t$  such that  $i \leq j$ , if Assumption 1 holds for both tasks, then for any policy  $\pi$  that serves task  $i$  but not  $j$  at  $(\mathbf{x}_t, s_t)$ , there exists a policy  $\bar{\pi}$  that serves task  $j$  but not  $i$  at  $(\mathbf{x}_t, s_t)$ , and results in the same or lower cost-to-go value, i.e., (19) holds.

The proof of Theorem 1 is shown in Appendix C.

*Remark 5:* Theorem 1 shows that under Assumption 1, the allocation of computing resource according to the ED-LDF order is without loss of optimality, regardless of future dynamics of the global state  $s_t$  (which determines renewable energy and task arrivals). In the heavy-traffic regime, the result of Theorem 1 does not require any information on the global state

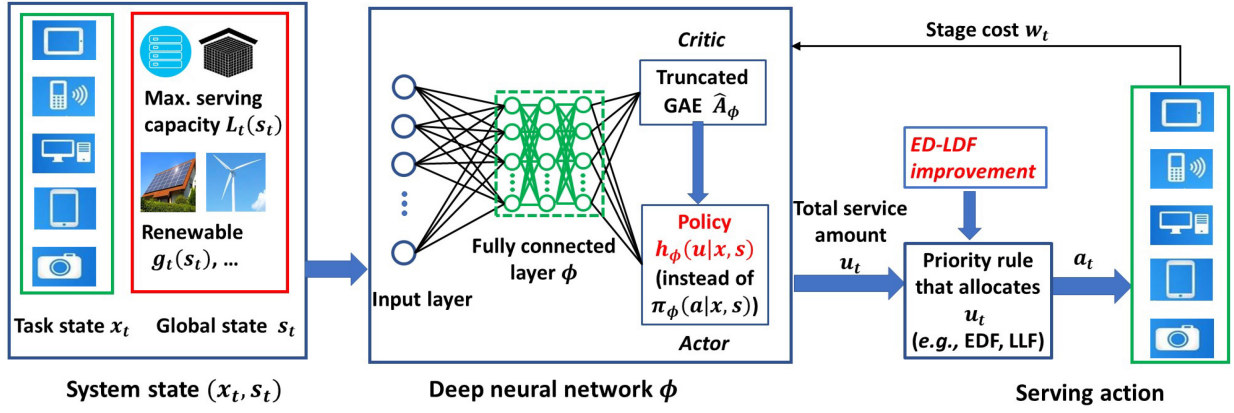


Fig. 3. Schematic overview of the proposed approach.

$s_t$ , which enables us to integrate the characterized ED-LDF order into model-free DRL methods.

It is worth noting that according to the ED-LDF order, within each deadline class, the tasks with more remaining demand will be served after finishing tasks with less remaining demand. Under Assumption 1, Theorem 1 guarantees that the cost savings from tasks with less remaining demand must outweigh the cost resulting from tasks with more remaining demand (that miss their deadlines or incur higher delay penalty).

We can use the (partial) ED-LDF order to improve the performance of an existing policy.

**Definition 3 (ED-LDF Improvement):** In preemptive scheduling, given a system state  $(\mathbf{x}_t, s_t)$  and the action  $\mathbf{a}_t$  taken by an arbitrary policy  $\pi$ , an ED-LDF improvement repeats the following process in each iteration until reaching the termination: if there exist a pair of tasks  $i < j$  with  $a_{i,t} = 1$ ,  $a_{j,t} = 0$ , then it interchanges the actions of  $i$ ,  $j$  (following the ED-LDF priority order); otherwise, the improvement process is terminated.

The optimality of ED-LDF improvement is established by Theorem 1 under mild assumptions. It is worth noting that the ED-LDF improvement always terminates in finite iterations.

**Lemma 3:** For any complete task order, the ED-LDF improvement process terminates in finite iterations.

**Proof:** According to Definitions 2 and 3, the positions of any pair of two tasks  $i$  and  $j$  will be interchanged at most once; hence, the ED-LDF improvement must terminate in finite iterations. ■

#### IV. ED-LDF POLICY-BASED PROXIMAL POLICY OPTIMIZATION APPROACH

In this section, we propose an ED-LDF-based PPO approach that combines the advantages of the data-driven DRL and optimal policy characterizations established in Theorem 1. In Section IV-A, we provide some background information on classical DRL methods. In Section IV-B, we introduce the proposed approach, discuss the advantage of using the truncated generalized advantage estimator (GAE), and describe the training of deep NNs (DNNs) in detail. In Section IV-C, we propose the ED-LDF-based PPO in Algorithm 1.

##### A. Preliminaries

DRL methods enhance classical reinforcement learning (RL) with DNNs for state representation or function approximation. Let  $\phi$  denote the set of parameters for DNN. As in [34], we consider a DNN architecture that shares the parameter  $\phi$  between the policy and the value function, i.e., the same  $\phi$  is used for both the critic and the actor networks.

Value-based DRL methods adopt DNNs to approximate the value function (through the critic network) [25]. We use  $V_\phi(\mathbf{x}_t, s_t)$  to denote the value function that is used to approximate the negative of the cost-to-go function  $J(\mathbf{x}_t, s_t)$ . The objective function (of the critic network) seeks to minimize the difference between  $V_\phi(\mathbf{x}_t, s_t)$  and  $-J(\mathbf{x}_t, s_t)$ . A widely adopted form is given by [26]

$$J^V(\phi) = \mathbb{E}_t \left\{ \left( V_\phi(\mathbf{x}_t, s_t) - (-J(\mathbf{x}_t, s_t)) \right)^2 \right\} = \mathbb{E}_t \{ J_t^V(\phi) \} \quad (23)$$

where expectation  $\mathbb{E}_t\{\cdot\}$  is the empirical average over a finite batch of samples in an algorithm that alternates between sampling and optimization, and  $(-J(\mathbf{x}_t, s_t))$  is the (unknown) optimal negative cost-to-go function that is estimated by different value-based RL methods.

##### B. ED-LDF-Based Proximal Policy Optimization

The proposed approach that integrates the ED-LDF characterization into PPO is depicted in Fig. 3. The stochastic policy  $h_\phi$  maps the current system state into the total service amount  $u_t$ , where  $h_\phi(u_t | \mathbf{x}_t, s_t)$  represents the probability of choosing the total service amount  $u_t$  at state  $(\mathbf{x}_t, s_t)$ . Compared with the classical PPO algorithm  $\pi_\phi(\mathbf{a}_t | \mathbf{x}_t, s_t)$ , the policy  $h_\phi(u_t | \mathbf{x}_t, s_t)$  significantly reduces the number of actions from  $2^{(M+N)}$  (with  $M + N$  being the maximum number of tasks in the serving system) to  $N$ . Based on the optimality result established in Theorem 1, the proposed approach applies the ED-LDF improvement on an existing priority rule to obtain the allocation action  $\mathbf{a}_t$  (cf. the right part of Fig. 3). Details of the proposed approach, such as the truncated GAE and structure of the fully connected layer  $\phi$  in Fig. 3 will be introduced in the remaining of this section.

Classical policy-based DRL methods compute an estimator of the policy gradient, which is useful in the stochastic gradient ascent algorithm. The estimator  $\nabla J_{PG}(\phi)$  with respect to the stochastic policy  $h_\phi$  is obtained by differentiating the objective function

$$J^{PG}(\phi) = \mathbb{E}_t \left\{ \log h_\phi(u_t | \mathbf{x}_t, s_t) \hat{A}_\phi(\mathbf{x}_t, s_t) \right\} \quad (24)$$

where  $h_\phi$  is the stochastic policy and  $\hat{A}_\phi(\mathbf{x}_t, s_t)$  is an estimator of the advantage function at  $(\mathbf{x}_t, s_t)$ .

Classical policy-based DRL methods, such as the REINFORCE [42], have two major disadvantages. First, the Monte Carlo sampling leads to high variance and slow learning speed. Second, the on-policy (training and sampling under the same policy) update usually converges to a local optimum.

To alleviate the first disadvantage, we consider a truncated version of GAE [33], [34] to obtain a good advantage function estimator with low variance

$$\hat{A}_\phi(\mathbf{x}_t, s_t) = \sum_{k=0}^{T-t} (\gamma \lambda)^k \delta_{t+k} \quad (25)$$

where  $\delta_t = -w_t(\mathbf{x}_t, s_t, \mathbf{a}_t) + \gamma V_\phi(\mathbf{x}_{t+1}, s_{t+1}) - V_\phi(\mathbf{x}_t, s_t)$  is the TD residual of  $V_\phi$  with discount  $\gamma$  [26], and the advantage parameter  $\lambda$  is used to adjust the bias-variance tradeoff.

To alleviate the second disadvantage, we introduce the state-of-the-art off-policy learning policy, the PPO proposed in [34], to enhance exploration. PPO considers the following clipped surrogate objective instead of  $J_{PG}(\phi)$  in (24):

$$\begin{aligned} J^{\text{CLIP}}(\phi) &= \mathbb{E}_t \left\{ \min \left( r_t(\phi) \hat{A}_\phi, \text{clip}(r_t(\phi), 1 - \epsilon, 1 + \epsilon) \hat{A}_\phi \right) \right\} \\ &= \mathbb{E}_t \left\{ J_t^{\text{CLIP}}(\phi) \right\} \end{aligned} \quad (26)$$

where  $\epsilon$  is a hyperparameter that controls the clip range,  $\hat{A}_\phi$  is the truncated GAE defined in (25), and

$$r_t(\phi) = \frac{h_\phi(u_t | \mathbf{x}_t, s_t)}{h_{\phi_{\text{old}}}(u_t | \mathbf{x}_t, s_t)}$$

is the policy probability ratio with  $\phi_{\text{old}}$  denoting the parameter of the sampling policy. The clip function

$$\text{clip}(r_t(\phi), 1 - \epsilon, 1 + \epsilon) = \begin{cases} 1 - \epsilon, & \text{if } r_t(\phi) \leq 1 - \epsilon \\ 1 + \epsilon, & \text{if } r_t(\phi) \geq 1 + \epsilon \\ r_t(\phi), & \text{otherwise} \end{cases}$$

in (26) avoids too large update from the sampling policy parameter  $\phi_{\text{old}}$  to the updated policy parameter  $\phi$ , by constraining  $r_t(\phi)$  inside the interval  $[1 - \epsilon, 1 + \epsilon]$  [34]. Since (26) takes the minimum of the clipped and unclipped objectives, the final objective  $J_{\text{CLIP}}(\phi)$  is a lower bound to the unclipped objective.

As in [34], we consider a parameter-shared DNN architecture that utilizes an overall objective (a combination of the clipped surrogate and the error term of the value network). According to (26) and (23), the overall maximization objective is

$$J^{\text{PPO}}(\phi) = \mathbb{E}_t \left\{ J_t^{\text{CLIP}}(\phi) - c J_t^V(\phi) \right\} \quad (28)$$

where  $c$  is the loss coefficient. Note that  $J_t^{\text{CLIP}}$  and  $J_t^V$  are the values obtained before taking the expectation [cf. the last term in (23) and (26)].

To construct the DNN with parameter  $\phi$ , we adopt the multilayer perceptron (MLP) architecture with three full connected layers of 128 units and hyperbolic tangent (tanh) activation function.<sup>3</sup> Such an architecture can uniformly approximate continuous functions [43].

### C. Algorithm

The proposed approach is summarized in Algorithm 1. In lines 1 and 2, we randomly initialize the DNN parameter  $\phi$  and  $\phi_{\text{old}}$  to obtain  $h_\phi$  for optimization and  $h_{\phi_{\text{old}}}$  for sampling. In each iteration, the algorithm alternates between sampling (lines 4–13) and optimization (lines 15–18). In lines 4–13,  $K$  parallel actors are used to accelerate the sampling process. In each state transition step, the algorithm first obtains  $u_t$  from the sampling policy  $h_{\phi_{\text{old}}}$  in lines 7 and 8. It then obtains the matching decision  $\ell_t$  by applying the result in Lemma 2 on an existing priority rule in line 9. In line 10, the serving vector  $\mathbf{a}_t$  is determined by applying the ED-LDF improvement on an existing priority rule. The corresponding system state transition and truncated GAE [defined in (25)] are recorded in the replay buffer (cf. line 11).

In each epoch of the optimization process, the DNN parameter  $\phi$  is updated by conducting stochastic gradient ascend (on the sampled data from the replay buffer  $\mathcal{B}$ ) with respect to the objective function (28) in lines 16 and 17. After the optimization stage, we update the sampling policy parameter  $\phi_{\text{old}}$  by  $\phi$  in line 19.

*Remark 6:* The proposed ED-LDF-based PPO approach has two main advantages. Compared with directly applying the state-of-the-art PPO algorithm, the dimensionality reduction in the action space (resulting from the ED-LDF-based priority rule) significantly improves the sample efficiency, particularly when the maximum number of tasks in the serving system  $M+N$  is large. As a result, the proposed approach learns faster and converges to better solutions (cf. the numerical results in Section V).

Second, by applying an optimal ED-LDF improvement on an existing priority rule, it is guaranteed by Theorem 1 that the proposed approach performs no worse than any existing priority rule under the heavy-traffic condition in Assumption 1.

*Remark 7:* Algorithm 1 is designed for both preemptive scheduling and nonpreemptive scheduling. For preemptive scheduling, given the aggregate serving amount  $u_t$ , the matching and serving decisions can be determined by  $\pi^P$  and  $\pi^A$  simultaneously in lines 9 and 10. For nonpreemptive scheduling, the matching policy  $\pi^{NP}$  needs to adopt a (possibly different) priority rule (that is improved by the results in Lemma 2) to make the matching decision  $\ell_t$  before implementing the allocation policy  $\pi^A$  in line 10.

## V. NUMERICAL RESULTS

For both preemptive and nonpreemptive scheduling, we apply the ED-LDF improvement (cf. Definition 3) on the

<sup>3</sup>The tanh function is given by  $\tanh(x) = (e^x - e^{-x}) / (e^x + e^{-x})$ .

**Algorithm 1** ED-LDF-Based PPO

---

```

1: Initialize  $\phi$  randomly to obtain  $h_\phi$ 
2: Initialize sampling policy  $h_{\phi_{old}}$  with  $\phi_{old} \leftarrow \phi$ 
3: for each iteration do
4:   for each (parallel) actor  $k = 1, \dots, K$  do
5:     Initialize an empty replay  $\mathcal{B}_k \leftarrow \emptyset$ 
6:     for each state transition step  $t = 0, \dots, T - 1$  do
7:       Find total service amount from policy:
8:        $u_t \sim h_{\phi_{old}}(u_t | \mathbf{x}_t, s_t)$ 
9:       Obtain  $\ell_t$  by applying the result in Lemma 2 on
       an existing priority rule.
10:      Obtain  $\mathbf{a}_t$  by applying ED-LDF improvement on
       an existing priority rule.
11:      Get  $(\mathbf{x}_{t+1}, s_{t+1})$ ,  $\hat{A}_\phi$  and record the transition:
        $\mathcal{B}_k \leftarrow \mathcal{B}_k \cup \{(\mathbf{x}_t, s_t), \mathbf{u}_t, -w(\mathbf{x}_t, s_t, \mathbf{a}_t), (\mathbf{x}_{t+1}, s_{t+1})\}$ 
12:    end for
13:  end for
14:  Cache all sampled data in the replay set  $\mathcal{B} = \bigcup_{k=1}^K \mathcal{B}_k$ .
15:  for each epoch do
16:    Update  $\phi$  w.r.t. the objective function (28), using the
    sampled data from  $\mathcal{B}$  for one epoch:
17:     $\phi \leftarrow \arg \max_{\phi} J^{\text{PPO}}(\phi)$ 
18:  end for
19:  Update  $h_{\phi_{old}}$  with  $\phi_{old} \leftarrow \phi$ 
20: end for

```

---

EDF rule as in line 10 in Algorithm 1, which is referred to as the PPO+ED-LDF approach. We numerically benchmark the proposed PPO+ED-LDF approach with conventional PPO, and three priority rule-based PPO approaches, i.e., PPO+EDF, PPO+LLF (least laxity first), and PPO+RR (round robin) [2]. In RR scheduling, tasks are queued in a first-in–first-out order, and each task is processed for one stage at its turn (if not finished, it will be put to the end of the queue). For preemptive scheduling, the same matching and allocation rule is used in all the aforementioned approaches. For nonpreemptive scheduling, the results in Lemma 2 are used to improve the matching decision.

#### A. Simulation Setup

We consider the preemptive and nonpreemptive scheduling of computation tasks faced by a cloud service provider over a period of 100 s ( $T = 100$ ). The cloud computing system contains  $N$  identical servers (i.e., virtual machines) and a public queue with a maximum length of  $M = 15N$ . From stage 0 to 45, the number of random arrival tasks follows the Poisson distribution with an arrival rate of  $N/3$ . No tasks arrive later than  $t = 45$ . It is worth noting that the remaining stages are long enough for the cloud service provider to fully serve all tasks before the final stage  $T = 100$ . For a fixed  $\beta = 0.4$ , we test the system of various scales, with  $M + N$  ranging from 320 to 640. It is worth noting that the nonconvexity induced by the  $O(M + N)$  binary variables (indicating whether each task is delayed or not) makes (one-shot or online) optimization-based approaches intractable for the scheduling problem even with  $M + N = 320$ .

The serving rate of each server is set as 0.1 GHz (i.e., 0.1 Gcycle/second). The initial states of arrival tasks are independent and identically distributed (i.i.d.) random variables with  $d_{i,t}$  (in computation time units) uniformly distributed over the set  $\{2, 3, \dots, 9, 10\}$ , i.e., the computation demand (on CPU cycles) of a newly arrived task is uniformly distributed over the set  $\{0.2, 0.3, \dots, 0.9, 1.0\}$  (in Gcycles) as in [40]. Each task  $i$  that arrives at (the beginning of) stage  $t$  has a deadline of  $t_i^d = d_{i,t} + t - 1$ . The aforementioned settings ensure that Assumption 1 almost always holds over the operating horizon.

The energy cost is linear in the total service amount as in [44], with  $F(u_t) = u_t$ . For a given global state  $s_t$ , the stage cost is the weighted sum of the energy cost (in kWh) and the delay cost (in second), i.e.,

$$w_t(\mathbf{x}_t, s_t, \mathbf{a}_t) = \beta \max(0, u_t - g_t(s_t)) + (1 - \beta)|\mathcal{D}_t|$$

where  $g_t(s_t)$  is the random renewable generation that follows a discrete time Markov chain and ranges from 0 to  $N/10$ . When the random renewable generation  $g_t(s_t)$  cannot meet the energy demand, the operator has to purchase electricity from the grid.

Fixing  $M + N = 480$ , we test the system with various  $\beta$  ranging from 0 to 0.8. When  $\beta = 0$ , the objective is to minimize the discounted delay cost  $\sum_t \gamma^t |\mathcal{D}_t|$ , which is equivalent to minimizing the discounted sum of the processing time of all delayed tasks.

For all the five approaches, we adopt the same MLP architecture (three fully connected layers of 128 units) with hyperbolic tangent (tanh) activation function to construct the DNN. We set the discount factor  $\gamma = 0.99$ , clipping parameter  $\epsilon = 0.2$ , and advantage parameter  $\lambda = 0.95$ . The other hyperparameters are set as default in [45]. For all the five approaches, we train three different instances with different random seeds and each training phase contains 10 000 episodes. The testing phase evaluates the average episode cost over 1000 simulations. All numerical experiments are implemented in Python 3.7 on an Intel Core i7-8565U (2 GHz) PC with 32 GB memory.

#### B. Performance Analysis Under Preemptive Scheduling

Under preemptive scheduling and  $\beta = 0.4$ , the training curves with  $M + N$  ranging from 320 to 640 are shown in Figs. 4–8. We note that given  $M + N$ , the training curves under different  $\beta$  settings have similar patterns.

For all the  $M + N$  settings in Figs. 4–8, PPO performs poorly due to the high dimensional action space. The proposed PPO+ED-LDF approach and three other priority rule-based PPO approaches learn much faster and converge to much better points, as they only need to output a scaler (the total serving amount) at each stage.

The performance gap between the the proposed PPO+ED-LDF approach and three other priority rule-based approaches demonstrates the power of the optimal ED-LDF improvement in the heavy traffic regime for preemptive scheduling. When  $M + N = 480$ , the performance of various approaches over different  $\beta$  settings is compared in Table I. Compared with PPO+LLF, PPO+RR, and PPO, the proposed PPO+ED-LDF

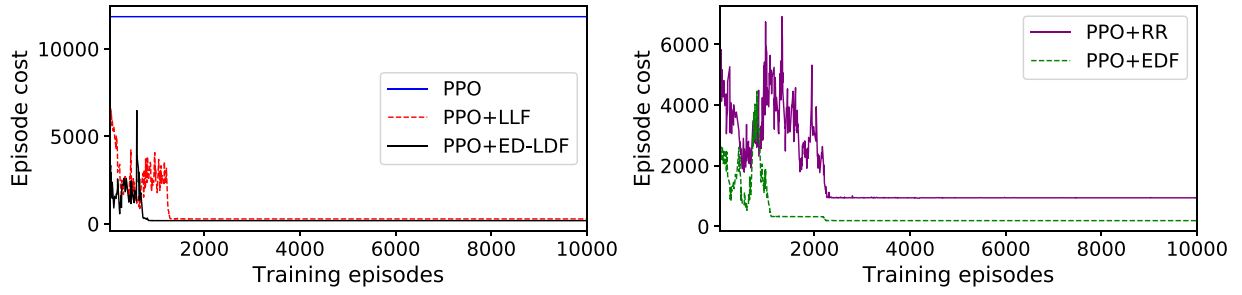


Fig. 4. Training curves with  $\beta = 0.4$  and  $M + N = 320$  under preemptive scheduling. The x-axis shows the total number of training episodes with each episode lasting 100 s. The y-axis shows the episode cost over the operation horizon. Each training curve is averaged over three different instances with different random seeds.

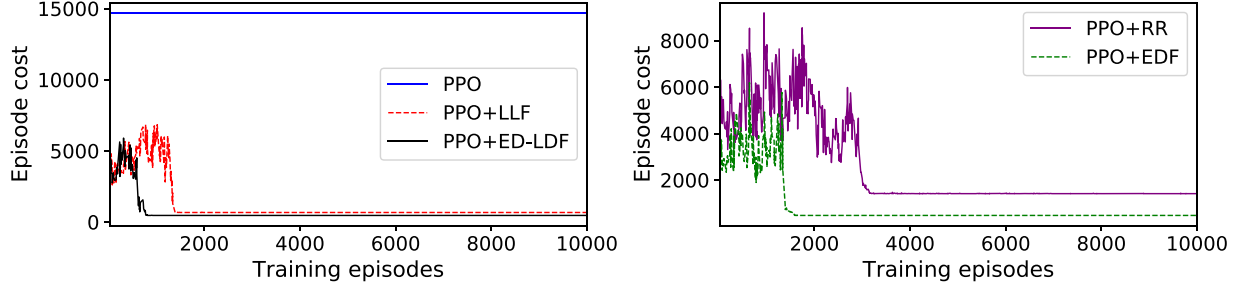


Fig. 5. Training curves with  $\beta = 0.4$  and  $M + N = 400$  under preemptive scheduling. The x-axis shows the total number of training episodes with each episode lasting 100 s. The y-axis shows the episode cost over the operation horizon. Each training curve is averaged over three different instances with different random seeds.

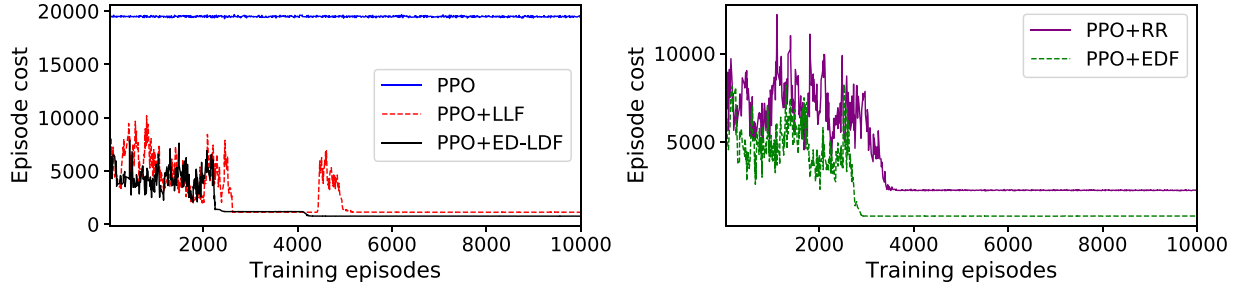


Fig. 6. Training curves with  $\beta = 0.4$  and  $M + N = 480$  under preemptive scheduling. The x-axis shows the total number of training episodes with each episode lasting 100 s. The y-axis shows the episode cost over the operation horizon. Each training curve is averaged over three different instances with different random seeds.

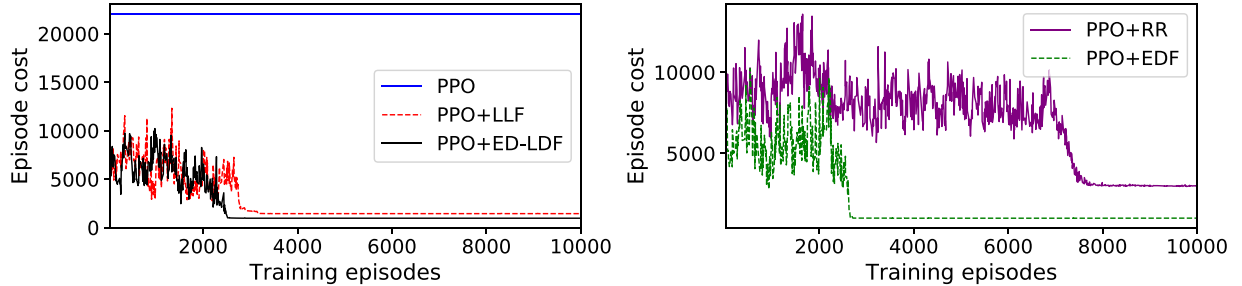


Fig. 7. Training curves with  $\beta = 0.4$  and  $M + N = 560$  under preemptive scheduling. The x-axis shows the total number of training episodes with each episode lasting 100 s. The y-axis shows the episode cost over the operation horizon. Each training curve is averaged over three different instances with different random seeds.

reduces 25%–34%, 53%–71%, and 94%–96% of the average total cost, respectively. Compared to PPO+EDF, the proposed PPO+ED-LDF approach reduces 3.2%–5.1% of the average cost. The percentage of cost reduction decreases with the parameter  $\beta$ , as the ED-LDF improvement mainly reduces the delay cost, and the weight of delay cost is  $1 - \beta$  [in (8)].

When  $\beta = 0.4$ , the performance of various approaches under different  $M + N$  settings is presented in Table II. The proposed PPO+ED-LDF approach achieves the best performance under all  $M + N$  settings. The proposed PPO+ED-LDF approach reduces 1.0% – 4.9%, 32% – 35%, 67% – 80% and 95% – 98% of the average total cost

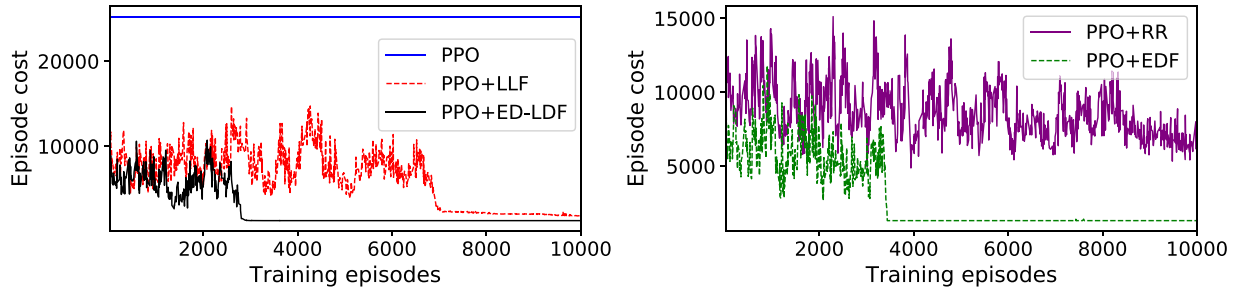


Fig. 8. Training curves with  $\beta = 0.4$  and  $M + N = 640$  under preemptive scheduling. The x-axis shows the total number of training episodes with each episode lasting 100 s. The y-axis shows the episode cost over the operation horizon. Each training curve is averaged over three different instances with different random seeds.

TABLE I  
AVERAGE TOTAL COST COMPARISON UNDER DIFFERENT  $\beta$  AND  
PREEMPTIVE SCHEDULING WITH  $M + N = 480$

| $\beta$    | 0       | 0.2     | 0.4     | 0.6     | 0.8    |
|------------|---------|---------|---------|---------|--------|
| PPO+ED-LDF | 1138.1  | 944.6   | 750.8   | 557.1   | 363.5  |
| PPO+EDF    | 1198.9  | 992.8   | 786.9   | 581.2   | 375.5  |
| PPO+LLF    | 1750.4  | 1434.9  | 1118.0  | 802.1   | 486.0  |
| PPO+RR     | 3860.8  | 2907.5  | 2222.6  | 1462.9  | 789.7  |
| PPO        | 31239.2 | 24991.4 | 18743.7 | 12495.6 | 6247.8 |

TABLE II  
AVERAGE TOTAL COST COMPARISON UNDER DIFFERENT  $M + N$  AND  
PREEMPTIVE SCHEDULING WITH  $\beta = 0.4$

| $M + N$    | 320     | 400     | 480     | 560     | 640     |
|------------|---------|---------|---------|---------|---------|
| PPO+ED-LDF | 184.0   | 456.3   | 750.9   | 960.2   | 1248.9  |
| PPO+EDF    | 186.2   | 473.8   | 786.9   | 1009.5  | 1304.6  |
| PPO+LLF    | 286.1   | 680.4   | 1118.1  | 1424.9  | 1848.5  |
| PPO+RR     | 916.8   | 1434.3  | 2222.6  | 3024.9  | 3804.6  |
| PPO        | 11840.1 | 14696.1 | 18743.6 | 22019.1 | 25235.7 |

TABLE III  
AVERAGE TOTAL COST COMPARISON UNDER DIFFERENT  $\beta$  AND  
NONPREEMPTIVE SCHEDULING WITH  $M + N = 480$

| $\beta$    | 0       | 0.2     | 0.4     | 0.6     | 0.8    |
|------------|---------|---------|---------|---------|--------|
| PPO+ED-LDF | 1192.2  | 989.3   | 786.6   | 583.7   | 381.0  |
| PPO+EDF    | 1258.2  | 1042.4  | 826.6   | 610.3   | 394.4  |
| PPO+LLF    | 2418.4  | 2019.2  | 1516.4  | 1067.6  | 618.8  |
| PPO+RR     | 3111.3  | 2520.2  | 1950.9  | 1341.2  | 750.7  |
| PPO        | 30436.5 | 24349.2 | 18261.9 | 12174.6 | 6087.2 |

TABLE IV  
AVERAGE TOTAL COST COMPARISON UNDER DIFFERENT  $M + N$  AND  
NONPREEMPTIVE SCHEDULING WITH  $\beta = 0.4$

| $M + N$    | 320     | 400     | 480     | 560     | 640     |
|------------|---------|---------|---------|---------|---------|
| PPO+ED-LDF | 196.3   | 479.1   | 786.6   | 1007.6  | 1529.9  |
| PPO+EDF    | 198.5   | 493.7   | 826.5   | 1059.2  | 1598.7  |
| PPO+LLF    | 428.9   | 918.2   | 1516.4  | 1914.0  | 2413.6  |
| PPO+RR     | 871.2   | 1362.3  | 1950.8  | 2347.3  | 2857.1  |
| PPO        | 11636.1 | 14735.7 | 18261.9 | 21238.0 | 24614.8 |

resulting from PPO+EDF, PPO+LLF, PPO+RR, and PPO, respectively.

For preemptive scheduling with  $M + N = 480$ , the training time of the proposed PPO+ED-LDF approach is around 2832 s, and the training time of PPO+EDF, PPO+LLF, PPO+RR and PPO is around 2725, 2826, 2814, and 4164 s, respectively. The scheduling overhead of a DRL model is mainly determined by the number of parameters in the DNN. Once a DRL model is trained, the average scheduling overhead in applying our DRL model (PPO+ED-LDF, PPO+EDF, PPO+LLF, PPO+RR, or PPO) is less than  $10^{-5}$  s in our numerical settings.

### C. Performance Analysis Under Nonpreemptive Scheduling

The training curves with  $\beta = 0.4$  and  $M + N = 480$  are shown in Fig. 9. We note that the training curves for nonpreemptive scheduling share similar patterns under different parameter settings. In Fig. 9, the PPO approach performs poorly under nonpreemptive scheduling due to the high dimensional action space. The other four priority rule-based PPO approaches learn faster and converge to better points, which is consistent with the results under preemptive scheduling (cf. Section V-B).

The performance gap between the the proposed PPO+ED-LDF approach and three other priority rule-based approaches demonstrates the power of the optimal ED-LDF improvement under heavy traffic and nonpreemptive scheduling. When  $M + N = 480$ , the performance of various approaches over different  $\beta$  settings is compared in Table III. Compared with PPO+LLF, PPO+RR, and PPO, the proposed PPO+ED-LDF approach reduces 38%–51%, 53%–70%, and 93%–96% of the average total cost, respectively. Compared to PPO+EDF, the proposed PPO+ED-LDF approach reduces 3.4%–5.1% of the average cost. The percentage of cost reduction decreases with the parameter  $\beta$  under nonpreemptive scheduling as well, as the proposed ED-LDF improvement mainly reduces the delay cost with weight  $1 - \beta$  [in (8)].

When  $\beta = 0.4$ , the performance of various approaches under different  $M + N$  settings for nonpreemptive scheduling is presented in Table IV. The proposed PPO+ED-LDF approach achieves the best performance under all  $M + N$  settings. The proposed PPO+ED-LDF approach reduces 1%–5%, 36%–54%, 46%–77%, and 93%–98% of the average total cost resulting from PPO+EDF, PPO+LLF, PPO+RR, and PPO, respectively.

For nonpreemptive scheduling with  $M + N = 480$ , the training time of the proposed PPO+ED-LDF approach is around

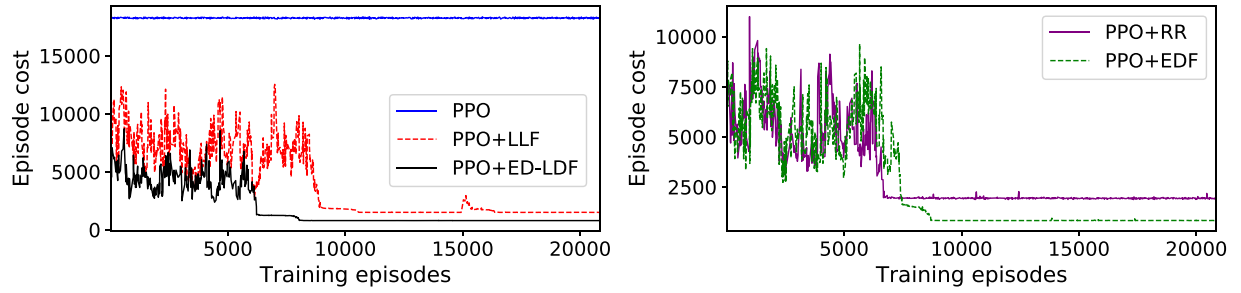


Fig. 9. Training curves with  $\beta = 0.4$  and  $M + N = 480$  under nonpreemptive scheduling. The x-axis shows the total number of training episodes with each episode lasting 100 s. The y-axis shows the episode cost over the operation horizon. Each training curve is averaged over three different instances with different random seeds.

TABLE V  
HIDDEN LAYER STRUCTURE OF THE CLASSIC AND DNNs WITH SAME NUMBER OF PARAMETERS

| $M + N$      | 320            | 400            | 480            | 560            | 640            |
|--------------|----------------|----------------|----------------|----------------|----------------|
| No. of Para. | 114k           | 135k           | 155k           | 176k           | 196k           |
| Deep NN      | $3 \times 128$ | $3 \times 128$ | $3 \times 128$ | $3 \times 128$ | $3 \times 128$ |
| Classic NN   | $1 \times 179$ | $1 \times 168$ | $1 \times 162$ | $1 \times 158$ | $1 \times 154$ |

TABLE VI  
AVERAGE TOTAL COST OF COMPARABLE DNN AND CLASSIC NN UNDER DIFFERENT  $M + N$  WITH  $\beta = 0.4$

| $M + N$                | 320   | 400   | 480   | 560    | 640    |
|------------------------|-------|-------|-------|--------|--------|
| Deep neural network    | 184.0 | 456.3 | 750.9 | 960.2  | 1248.9 |
| Classic neural network | 184.8 | 456.4 | 751.8 | 1203.8 | 1576.8 |

5653 s, and the training time of PPO+EDF, PPO+LLF, PPO+RR and PPO is around 5512, 5439, 5473, and 7210 s, respectively. The average scheduling overhead in applying the DRL model (PPO+ED-LDF, PPO+EDF, PPO+LLF, PPO+RR or PPO) is less than  $10^{-5}$  s in our numerical settings.

#### D. Comparison Between Deep Neural Network and Classic Neural Network

The performance of an NN highly depends on its network structure, and it is reported that the deep multilayer structure works better than its one-layer counterpart in some applications [46]. However, to the best of our knowledge, the performance of these two structures has not been compared for the considered scheduling problem in cloud computing. In this section, we compare the performance of the proposed PPO+ED-LDF approach for preemptive scheduling with two different NN structures: 1) a DNN with three full connected layers of 128 units ( $3 \times 128$ ) and 2) a classic NN with one full connected layer. To make a fair comparison, we let the classic NN have (roughly) the same number of parameters as their deep NN counterparts as shown in Table V.

The average total cost of the two different NN structures under different parameter settings is compared in Tables VI and VII. We note that deep NN outperforms classic NN by 0%–20%, which is consistent with the result in [46]. From Table VI, we note that the performance gap between deep and classic NNs increases with  $M + N$  when  $\beta = 0.4$ .

TABLE VII  
AVERAGE TOTAL COST OF COMPARABLE DNN AND CLASSIC NN UNDER DIFFERENT  $\beta$  WITH  $M + N = 480$

| $\beta$    | 0      | 0.2   | 0.4   | 0.6   | 0.8   |
|------------|--------|-------|-------|-------|-------|
| Deep NN    | 1138.1 | 944.6 | 750.8 | 557.1 | 363.5 |
| Classic NN | 1138.2 | 944.8 | 751.8 | 557.2 | 363.5 |

## VI. CONCLUSION

Through a dynamic programming formulation, we studied the scheduling of multiple tasks in a serving system with  $N$  servers and a public queue of  $M$  positions, under stochastic renewable generation and task arrivals. The formulated dynamic program is intractable when the maximum number of tasks in the serving system  $M + N$  is large, due to the well-known curse of dimensionality in the action space.

For the heavy traffic regime, we established a new, optimal, partial priority order (ED-LDF): priority should be given to tasks with earlier deadline and less demand. The characterized ED-LDF order is shown to be optimal regardless of the dynamics of system uncertainties, which enables us to integrate it into (model-free) DRL methods, reducing the outputs of DNNs to a scalar, i.e., the total service amount. As such, the proposed ED-LDF-based PPO approach is scalable with respect to the maximum number of tasks in the serving system. Numerical results demonstrate that the proposed approach outperforms the classical PPO method and three priority rule-based PPO approaches, PPO+EDF (earliest deadline first), PPO+LLF (least laxity first), and PPO+RR (round robin) under various parameter settings.

## APPENDIX A PROOF OF LEMMA 1

For an arbitrary-realized global state trajectory  $\{s_k\}_{k=t}^{T-1}$ , let  $\{a_k\}_{k=t}^{T-1}$  (with  $a_{i,t} = 1$  and  $a_{j,t} = 0$ ) denote the actions under policy  $\pi$  with allocation policy  $\pi^A$ . Let  $\{\mathbf{x}_k\}_{k=t+1}^T$  be the corresponding states resulting from policy  $\pi$  and  $\{s_k\}_{k=t}^{T-1}$ .

We will construct a policy  $\tilde{\pi}$  with allocation policy  $\tilde{\pi}^A$  to prove Lemma 1. Let  $\{\tilde{a}_k\}_{k=t}^{T-1}$  be the actions taken by the policy  $\tilde{\pi}$  and  $\{\tilde{\mathbf{x}}_k\}_{k=t+1}^T$  be the resulting states. At stage  $t$ , we let  $\tilde{a}_t = a_t$  except that

$$\tilde{a}_{i,t} = a_{j,t} = 0, \quad \tilde{a}_{j,t} = a_{i,t} = 1. \quad (29)$$

Let  $t_i$  and  $t_j$  denote the stages where tasks  $i$  and  $j$  leave the serving system under policy  $\pi$ , respectively.

If  $t_i \geq t_j$ , it follows from  $d_{i,t} = d_{j,t}$  and (29) that there exists a stage  $\bar{t} < \min\{t_i, t_j\}$  such that:

$$\sum_{k=\bar{t}}^{\bar{t}} \bar{a}_{i,k} = \sum_{k=\bar{t}}^{\bar{t}} a_{i,k} \quad (30)$$

under policy  $\pi$ . Under policy  $\bar{\pi}$ , it is therefore feasible to let

$$\bar{a}_{i,k} = a_{j,k}, \bar{a}_{j,k} = a_{i,k} \quad \forall k \in \{t, \dots, \bar{t}\}. \quad (31)$$

We can construct a feasible sequence of actions by letting  $\{\bar{a}_k\}_{k=t}^{T-1} = \{a_k\}_{k=t}^{T-1}$ , except the actions specified in (31). According to (8) and (31), we have

$$w_k(\bar{x}_k, s_k, \bar{a}_k) = w_k(x_k, s_k, a_k) \quad \forall k \in \{t, \dots, T-1\}. \quad (32)$$

We now consider the case with  $t_i < t_j$ . Since  $d_{i,t} = d_{j,t}$ , we can construct a feasible sequence of actions  $\{\bar{a}_k\}_{k=t}^{T-1}$  such that

$$\bar{a}_{i,k} = a_{j,k} \quad \forall k \in \{t+1, \dots, t_j\} \quad (33)$$

and

$$\bar{a}_{j,k} = a_{i,k} \quad \forall k \in \{t+1, \dots, t_i\}. \quad (34)$$

Note that under (33) and (34), tasks  $i$  and  $j$  leave the serving system at  $t_j$  and  $t_i$ , respectively.

Note that in the case with  $t_i < t_j$ , the index available under policy  $\pi$  may not be available under policy  $\bar{\pi}$ . For example, we can index a newly arrived task as  $i$  at stage  $t_i + 1$  under policy  $\pi$ , but cannot index it as  $i$  under policy  $\bar{\pi}$  constructed in (33). However, in such a case, we can index it as  $j$  under policy  $\bar{\pi}$ . We will therefore interchange the indices  $i$  and  $j$  for the tasks arriving after stage  $\min\{t_i, t_j\}$ , and set

$$\bar{a}_{i,k} = a_{j,k} \quad \forall k \in \{t_j + 1, \dots, T-1\} \quad (35)$$

and

$$\bar{a}_{j,k} = a_{i,k} \quad \forall k \in \{t_i + 1, \dots, T-1\}. \quad (36)$$

By interchanging the indices  $i$  and  $j$  for tasks that arrive after stage  $\min\{t_i, t_j\}$ , we can construct a feasible sequence of actions by letting  $\{\bar{a}_k\}_{k=t}^{T-1} = \{a_k\}_{k=t}^{T-1}$  except the actions specified in (33)–(36).

According to (8) and (33)–(36),  $\bar{a}_k$  results in the same serving cost and the same or less delay cost than  $a_k$  since  $t_i^d > t_j^d$ , and we have

$$w_k(\bar{x}_k, s_k, \bar{a}_k) \leq w_k(x_k, s_k, a_k) \quad \forall k \in \{t, \dots, T-1\}. \quad (37)$$

At stage  $k$  with  $t_i < k \leq t_j$  and  $t_i^d \geq k > t_j^d$ , the actions defined in (33)–(36) would reduce the delay cost, and as a result, the inequality in (37) strictly holds [cf. the expression of stage cost in (8)].

Since the global state realization  $\{s_k\}_{k=t}^{T-1}$  is arbitrarily chosen, according to (8) and (16), the policy  $\bar{\pi}$  constructed above satisfies (19) and, thus, Lemma 1 holds.

## APPENDIX B PROOF OF LEMMA 2

In the nonpreemptive scheduling case, we consider an arbitrary-realized global state trajectory  $\{s_k\}_{k=t}^{T-1}$ , and let  $\{a_k\}_{k=t}^{T-1}$  be the actions under policy  $\pi$ , leading to  $i \in \mathcal{L}_t$  and  $j \notin \mathcal{L}_t$  at stage  $t$ . Let  $\{\bar{a}_k\}_{k=t}^{T-1}$  be the actions taken by policy  $\bar{\pi}$ ,  $\{\bar{x}_k\}_{k=t+1}^T$ ,  $\{\bar{\mathcal{L}}_k\}_{k=t+1}^T$ , and  $\{\bar{\mathcal{T}}_k\}_{k=t+1}^T$  be the resulting sequence of states, the set of matched tasks, and the set of arrived tasks, respectively.

We will construct a policy  $\bar{\pi}$  to prove the lemma. Let  $\{\bar{a}_k\}_{k=t}^{T-1}$  be the actions taken by the policy  $\bar{\pi}$  and  $\{\bar{x}_k\}_{k=t+1}^T$  be the resulting states. At stage  $t$ , the matching policy  $\bar{\pi}^{NP}$  makes the same matching decision except for task  $i, j$ , i.e.,  $i \notin \bar{\mathcal{L}}_t$  and  $j \in \bar{\mathcal{L}}_t$ . At stages after  $t$ , for tasks other than  $i, j$ , the matching policy  $\bar{\pi}^{NP}$  always makes the same matching decision as  $\pi^{NP}$ . In what follows, we focus on the matching decisions on tasks  $i, j$ .

Let  $t_i$  and  $t_j$  denote the stages when tasks  $i$  and  $j$  leave the serving system under policy  $\pi$ , respectively.

Let  $t' > t$  be the first stage when  $\pi^{NP}$  matches task  $j$  to the server. Note that under policy  $\pi$  and nonpreemptive scheduling, we have  $j \in \mathcal{L}_k$  for all  $k \in \{t', \dots, t_j\}$ . We let  $\bar{\pi}^{NP}$  make the same matching decision as  $\pi^{NP}$  except for tasks  $i$  and  $j$  before  $t'$ , i.e.,

$$i \in \mathcal{L}_k, j \notin \mathcal{L}_k; i \notin \bar{\mathcal{L}}_k, j \in \bar{\mathcal{L}}_k \quad \forall k \in \{t, \dots, t' - 1\}. \quad (38)$$

We let stage  $t'$  be the first stage when  $\bar{\pi}^{NP}$  matches  $i$  to the server, i.e.,  $i \notin \bar{\mathcal{L}}_{t'-1}$  and  $i \in \bar{\mathcal{L}}_{t'}$ .

Note that under both matching policies, tasks  $i, j$  are matched to the server at stage  $t'$ , i.e.,

$$i \in \mathcal{L}_{t'}, j \in \mathcal{L}_{t'}; i \in \bar{\mathcal{L}}_{t'}, j \in \bar{\mathcal{L}}_{t'}. \quad (39)$$

Under nonpreemptive scheduling, tasks  $i$  and  $j$  will be matched to the server until they leave the serving system; thus, for policy  $\bar{\pi}$ , we have

$$i \in \bar{\mathcal{L}}_k, \text{ if } i \in \bar{\mathcal{T}}_k; j \in \bar{\mathcal{L}}_k, \text{ if } j \in \bar{\mathcal{T}}_k \\ \forall k \in \{t' + 1, \dots, T-1\} \quad (40)$$

where  $\bar{\mathcal{T}}_k$  is determined by  $(\bar{x}_{k-1}, s_{k-1})$  and  $\bar{a}_{k-1}$ .

In what follows, given the matching policy  $\bar{\pi}^{NP}$  constructed in (38)–(40), we construct a sequence of actions  $\{\bar{a}_k\}_{k=t}^{T-1}$  for the allocation policy  $\bar{\pi}^A$ .

Assume that under policy  $\pi$ , both  $i$  and  $j$  are not served from stage  $t$  to  $t'$ , that is

$$a_{i,k} = a_{j,k} = 0 \quad \forall k \in \{t, \dots, t'\}. \quad (41)$$

According to (38)–(40), it is feasible to let  $\{\bar{a}_k\}_{k=t}^{T-1} = \{a_k\}_{k=t}^{T-1}$ . According to (8), we have (32).

If (41) does not hold and  $t_i \geq t_j$ , then according to (38)–(40), it is feasible to let  $\{\bar{a}_k\}_{k=t}^{T-1} = \{a_k\}_{k=t}^{T-1}$  except the actions specified in (31). According to (8) and (31), again we have (32).

If (41) does not hold and  $t_i < t_j$ , then according to (38)–(40), it is feasible to let  $\{\bar{a}_k\}_{k=t}^{T-1} = \{a_k\}_{k=t}^{T-1}$  except the actions specified in (33)–(36). According to (8) and (33)–(36), we have (37).

Since the global state realization  $\{s_k\}_{k=t}^{T-1}$  is arbitrarily chosen, according to (8), the policy  $\bar{\pi}$  constructed above satisfies (19) and, thus, Lemma 2 holds.

#### APPENDIX C PROOF OF THEOREM 1

The following lemma will be useful in the proof of Theorem 1.

**Lemma 4:** In preemptive scheduling, given a system state  $(\mathbf{x}_t, s_t)$  with two tasks  $i, j$  such that  $d_{i,t} > d_{j,t}$  and  $\tau_{i,t} = \tau_{j,t}$ , if Assumption 1 holds for both tasks  $i$  and  $j$ , then for any policy  $\pi$  that serves task  $i$  but not  $j$  at  $(\mathbf{x}_t, s_t)$ , there exists a policy  $\bar{\pi}$  that serves task  $j$  but not  $i$  at  $(\mathbf{x}_t, s_t)$ , and results in the same or lower cost-to-go value, i.e., (19) holds.

*Proof:* For an arbitrary global states trajectory  $\{s_k\}_{k=t}^{T-1}$ , let  $\{a_k\}_{k=t}^{T-1}$  be the actions taken by policy  $\pi$  (with  $a_{i,t} = 1$  and  $a_{j,t} = 0$ ). Let  $\{\mathbf{x}_k\}_{k=t+1}^T$  be the corresponding states resulting from policy  $\pi$  and  $\{s_k\}_{k=t}^{T-1}$ . We will construct a policy  $\bar{\pi}$  to prove Lemma 4. Let  $\{\bar{a}_k\}_{k=t}^{T-1}$  be the actions taken by the policy  $\bar{\pi}$  and  $\{\bar{\mathbf{x}}_k\}_{k=t+1}^T$  be the resulting states. At stage  $t$ , we let  $\bar{a}_t = a_t$  except that

$$\bar{a}_{i,t} = 0, \bar{a}_{j,t} = 1. \quad (42)$$

Let  $t_i$  and  $t_j$  denote the stages where tasks  $i$  and  $j$  leave the serving system under policy  $\pi$ , respectively. Let  $\bar{t}_i$  and  $\bar{t}_j$  denote the stages where tasks  $i$  and  $j$  leave the serving system under policy  $\bar{\pi}$ , respectively.

We consider two different cases. First, when there exists a stage  $\bar{t} \leq t_j$  such that

$$\sum_{k=t}^{\bar{t}} a_{i,k} = \sum_{k=t}^{\bar{t}} a_{j,k} \quad (43)$$

since  $d_{i,t} > d_{j,t}$ , it is feasible to set  $\{\bar{a}_k\}_{k=t}^{T-1} = \{a_k\}_{k=t}^{T-1}$  except

$$\bar{a}_{i,k} = a_{j,k}, \bar{a}_{j,k} = a_{i,k} \quad \forall k \in \{t, \dots, \bar{t}\}. \quad (44)$$

In this case, according to (43) and (44), we have  $\bar{t}_i = t_i$  and  $\bar{t}_j = t_j$ .

According to (8) and (44),  $\{\bar{a}_k\}_{k=t}^{T-1}$  and  $\{a_k\}_{k=t}^{T-1}$  result in the same stage cost from  $t$  to  $T-1$ , leading to (32) in this case.

Second, if (43) does not hold for any  $\bar{t} \leq t_j$ , then it follows from  $a_{i,t} = 1$  and  $a_{j,t} = 0$  that there exists a stage  $t' < t_j$  such that:

$$\sum_{k=t}^{t'} a_{i,k} = d_{j,t}. \quad (45)$$

In this case, it is feasible to set

$$\bar{a}_{i,k} = a_{j,k}, \bar{a}_{j,k} = a_{i,k} \quad \forall k \in \{t+1, \dots, t'\} \quad (46)$$

which finishes task  $j$  at stage  $t'$  (i.e.,  $\bar{t}_j = t'$ ), reducing the delay cost incurred by task  $j$  from stage  $t' + 1$  to  $t_j$ .

For stages  $k \in \{t' + 1, \dots, t_j\}$ , however, it is infeasible to set  $\bar{a}_{j,k} = a_{j,k}$  when  $a_{j,k} = 1$ , since task  $j$  has been completed at stage  $t'$  according to (42) and (46). We therefore set

$$\bar{a}_{j,k} = 0, k \in \{t' + 1, \dots, t_j\}. \quad (47)$$

Note that (47) may lead to

$$\bar{a}_{j,k} = a_{j,k} - 1 \quad (48)$$

for some stage  $k \in \{t' + 1, \dots, t_j\}$ . Without loss of generality, we assume  $t_j$  is the only stage when (48) holds. If there exists two or more stages in  $\{t' + 1, \dots, t_j\}$  such that (48) holds, we can repeat the following process to prove the desired result.

According to (48), under the policy  $\bar{\pi}$  that takes the same actions as policy  $\pi$  except on task  $j$  at stage  $t_j$ , the total service amount may decrease at stage  $t_j$ , i.e.,  $\bar{u}_{t_j} = u_{t_j} - 1$ , where  $\bar{u}_{t_j}$  is the total service amount at stage  $t_j$  under policy  $\bar{\pi}$ .

In addition, it is feasible to let

$$\bar{a}_{i,k} = a_{i,k} \quad \forall k \in \{t' + 1, \dots, t_i\} \quad (49)$$

according to (42) and (46). Note that from  $d_{i,t} > d_{j,t}$  and (45), we have  $t' < t_i$ .

Without loss of generality, we assume that no arrival task will use index  $i$  after stage  $t_i$ . According to (42), (46), and (49), task  $i$  is not completed at stage  $t_i$  under policy  $\bar{\pi}$ , and needs to be served at stage  $u_{t_i+1}$ . Since  $u_{t_i+1} > 0$  according to the fact that Assumption 1 holds for tasks  $i$  and  $j$ , and  $t_i^{\max}(\mathbf{x}_t, s_t) \geq t_i + 1$  according to the definition of  $t_i^{\max}$  in (22), it is feasible to set

$$\bar{a}_{i,t_i+1} = 1 \quad (50)$$

under (46) and (49). According to (46), (49), and (50), we have  $\bar{t}_i = t_i + 1$ .<sup>4</sup> Note that according to (50), under the policy  $\bar{\pi}$  that takes the same actions on tasks other than  $i$  as policy  $\pi$  at stage  $t_i$ , the total service amount may increase at stage  $t_i$ , i.e.,  $\bar{u}_{t_i} = u_{t_i} + 1$ .

When  $t_j \neq t_i + 1$ , to maintain the total service amount and the serving cost unchanged at stages  $t_j$  and  $t_i + 1$  under the constructed policy  $\bar{\pi}$ , we need to adjust the actions on a set of tasks other than  $i$  and  $j$ .

Since for any  $k \in \{t, \dots, \max[t_i + 1, t_j]\}$ , we have  $|i \mid d_{i,k} \geq 1, i \in \mathcal{I}_k| > C_k(s_k)$  and  $u_k > 0$  according to the fact that Assumption 1 holds for tasks  $i, j$ . Hence, there must exist a set of tasks  $\{m_1, \dots, m_{\max}\}$  such that

$$\begin{aligned} a_{m_1,t_j} &= 0, a_{m_1,t_i} = 1, a_{m_2,t_1} = 0, a_{m_2,t_2} = 1, \dots \\ a_{m_{\max},t_{\max-1}} &= 0, a_{m_{\max},t_i+1} = 1 \end{aligned} \quad (51)$$

under policy  $\pi$ , where  $t_j < t_1 < \dots < t_i + 1$  or  $t_j > t_1 > \dots > t_i + 1$ . The set of tasks to be adjusted is empty if and only if  $t_j = t_i + 1$ . According to (47) and (50), it is feasible to set

$$\begin{aligned} \bar{a}_{m_1,t_j} &= 1, \bar{a}_{m_1,t_i} = 0, \bar{a}_{m_2,t_1} = 1, \bar{a}_{m_2,t_2} = 0, \dots \\ \bar{a}_{m_{\max},t_{\max-1}} &= 1, \bar{a}_{m_{\max},t_i+1} = 0 \end{aligned} \quad (52)$$

which ensures that the policy  $\bar{\pi}$  must lead to the same serving cost as the original policy  $\pi$ .

As a result, it is feasible to set  $\{\bar{a}_k\}_{k=t}^{T-1} = \{a_k\}_{k=t}^{T-1}$  except the actions specified in (42), (46), (47), (50), and (52). Since  $\theta_{j,t} \leq 0, \theta_{i,t} \leq 0$  (from Assumption 1) and  $\tau_{i,t} = \tau_{j,t}$ , the action sequence  $\{\bar{a}_k\}_{k=t}^{T-1}$  constructed in (42), (46), (47), (50),

<sup>4</sup>If (48) holds for  $r > 1$  stages in  $\{t' + 1, \dots, t_j\}$ , then it is feasible to let  $\bar{a}_{i,t_i+1} = 1, \dots, \bar{a}_{i,t_i+r} = 1$ , and we have  $\bar{t}_i = t_i + r$ .

and (52) results in the same serving cost, from  $t$  to  $T-1$  and the same or less accumulated delay cost than  $\{a_k\}_{k=t}^{T-1}$  from  $t$  to  $T-1$ , that is

$$\sum_{k=t}^{T-1} \gamma^{k-t} w_k(\bar{x}_k, s_k, \bar{a}_k) \leq \sum_{k=t}^{T-1} \gamma^{k-t} w_k(x_k, s_k, a_k). \quad (53)$$

In particular, the inequality in (53) strictly holds when  $\gamma < 1$  and  $t_j - t' > 1$  according to (8) and (16).<sup>5</sup>

Since the global state realization  $\{s_k\}_{k=t}^{T-1}$  is arbitrarily chosen, according to (8) and (16), the policy  $\bar{\pi}$  constructed above satisfies (19) and, thus, Lemma 4 holds. ■

According to Definition 2,  $i \leq j$  implies that  $d_{i,t} \geq d_{j,t}$  and  $t_i^d \geq t_j^d$ . We argue that Theorem C holds in the following three different cases.

If  $d_{i,t} = d_{j,t}$  and  $\tau_{i,t} \geq \tau_{j,t}$ , then it follows from Lemmas 1, and 2 that (19) holds.

If  $d_{i,t} \geq d_{j,t}$  and  $\tau_{i,t} = \tau_{j,t}$ , then it follows from Lemma 4 that (19) holds.

If  $d_{i,t} > d_{j,t}$ , and  $t_i^d > t_j^d$ , then it follows from Lemmas 1, 2, and 4 that (19) holds. This completes the proof of Theorem 1.

## REFERENCES

- [1] N. Kumar, G. S. Aujla, S. Garg, K. Kaur, R. Ranjan, and S. K. Garg, "Renewable energy-based multi-indexed job classification and container management scheme for sustainability of cloud data centers," *IEEE Trans. Ind. Informat.*, vol. 15, no. 5, pp. 2947–2957, May 2019.
- [2] G. S. Aujla, M. Singh, N. Kumar, and A. Y. Zomaya, "Stackelberg game for energy-aware resource allocation to sustain data centers using RES," *IEEE Trans. Cloud Comput.*, vol. 7, no. 4, pp. 1109–1123, Oct.–Dec. 2019.
- [3] M. Chen, M. Dong, and B. Liang, "Resource sharing of a computing access point for multi-user mobile cloud offloading with delay constraints," *IEEE Trans. Mobile Comput.*, vol. 17, no. 12, pp. 2868–2881, Dec. 2018.
- [4] W. Zhan *et al.*, "Deep-reinforcement-learning-based offloading scheduling for vehicular edge computing," *IEEE Internet Things J.*, vol. 7, no. 6, pp. 5449–5465, Jun. 2020.
- [5] X. Xiong, K. Zheng, L. Lei, and L. Hou, "Resource allocation based on deep reinforcement learning in IoT edge computing," *IEEE J. Sel. Areas Commun.*, vol. 38, no. 6, pp. 1133–1146, Jun. 2020.
- [6] R. Deng, R. Lu, C. Lai, T. H. Luan, and H. Liang, "Optimal workload allocation in fog-cloud computing toward balanced delay and power consumption," *IEEE Internet Things J.*, vol. 3, no. 6, pp. 1171–1181, Dec. 2016.
- [7] E. Fountoulakis, N. Pappas, Q. Liao, A. Ephremides, and V. Angelakis, "Dynamic power control for packets with deadlines," in *Proc. IEEE Global Commun. Conf. (GLOBECOM)*, 2018, pp. 1–6.
- [8] F. Shan, J. Luo, W. Wu, M. Li, and X. Shen, "Discrete rate scheduling for packets with individual deadlines in energy harvesting systems," *IEEE J. Sel. Areas Commun.*, vol. 33, no. 3, pp. 438–451, Mar. 2015.
- [9] X. Wang, Y. Pi, and A. Tang, "Scheduling of electric vehicle charging via multi-server fair queueing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 28, no. 11, pp. 3298–3312, Nov. 2017.
- [10] B. Wan, J. Dang, Z. Li, H. Gong, F. Zhang, and S. Oh, "Modeling analysis and cost-performance ratio optimization of virtual machine scheduling in cloud computing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 31, no. 7, pp. 1518–1532, Jul. 2020.
- [11] H. Wu and K. Wolter, "Stochastic analysis of delayed mobile offloading in heterogeneous networks," *IEEE Trans. Mobile Comput.*, vol. 17, no. 2, pp. 461–474, Feb. 2018.
- [12] C. L. Liu and J. W. Layland, "Scheduling algorithms for multiprogramming in a hard-real-time environment," *J. ACM*, vol. 20, no. 1, pp. 46–61, 1973.
- [13] S.-H. Oh and S.-M. Yang, "A modified least-laxity-first scheduling algorithm for real-time tasks," in *Proc. 5th Int. Conf. Real-Time Comput. Syst. Appl.*, 1998, pp. 31–36.
- [14] E. Bitar and Y. Xu, "Deadline differentiated pricing of deferrable electric loads," *IEEE Trans. Smart Grid*, vol. 8, no. 1, pp. 13–25, Jan. 2017.
- [15] C. D. Locke, "Best-effort decision-making for real-time scheduling," Ph.D. dissertation, Dept. Comput. Sci., Carnegie Mellon Univ., Pittsburgh, PA, USA, 1987.
- [16] Z. Yu, Y. Xu, and L. Tong, "Deadline scheduling as restless bandits," *IEEE Trans. Autom. Control*, vol. 63, no. 8, pp. 2343–2358, Aug. 2018.
- [17] Y. Xu, P. Cheng, Z. Chen, M. Ding, Y. Li, and B. Vucetic, "Task offloading for large-scale asynchronous mobile edge computing: An index policy approach," *IEEE Trans. Signal Process.*, vol. 69, pp. 401–416, 2021. [Online]. Available: <https://ieeexplore.ieee.org/document/9303388>
- [18] H.-S. Lee and J.-W. Lee, "Resource and task scheduling for SWIPT IoT systems with renewable energy sources," *IEEE Internet Things J.*, vol. 6, no. 2, pp. 2729–2748, Apr. 2019.
- [19] N. Eshraghi and B. Liang, "Joint offloading decision and resource allocation with uncertain task computing requirement," in *Proc. IEEE Conf. Comput. Commun. (INFOCOM)*, 2019, pp. 1414–1422.
- [20] M. H. Chen, B. Liang, and M. Dong, "Multi-user multi-task offloading and resource allocation in mobile cloud systems," *IEEE Trans. Wireless Commun.*, vol. 17, no. 10, pp. 6790–6805, Oct. 2018.
- [21] L. P. Qian, B. Shi, Y. Wu, B. Sun, and D. H. K. Tsang, "NOMA-enabled mobile edge computing for Internet of Things via joint communication and computation resource allocations," *IEEE Internet Things J.*, vol. 7, no. 1, pp. 718–733, Jan. 2020.
- [22] Y. Mao, J. Zhang, and K. B. Letaief, "Dynamic computation offloading for mobile-edge computing with energy harvesting devices," *IEEE J. Sel. Areas Commun.*, vol. 34, no. 12, pp. 3590–3605, Dec. 2016.
- [23] Y. Liu, S. Xie, Q. Yang, and Y. Zhang, "Joint computation offloading and demand response management in mobile edge network with renewable energy sources," *IEEE Trans. Veh. Technol.*, vol. 69, no. 12, pp. 15720–15730, Dec. 2020.
- [24] G. Lee, W. Saad, and M. Bennis, "An online optimization framework for distributed fog network formation with minimal latency," *IEEE Trans. Wireless Commun.*, vol. 18, no. 4, pp. 2244–2258, Apr. 2019.
- [25] V. Mnih *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.
- [26] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*. Cambridge, MA, USA: MIT Press, 2018.
- [27] S. Huang, B. Lv, R. Wang, and K. Huang, "Scheduling for mobile edge computing with random user arrivals—An approximate MDP and reinforcement learning approach," *IEEE Trans. Veh. Technol.*, vol. 69, no. 7, pp. 7735–7750, Jul. 2020.
- [28] J. Xu, L. Chen, and S. Ren, "Online learning for offloading and autoscaling in energy harvesting mobile edge computing," *IEEE Trans. Cogn. Commun. Netw.*, vol. 3, no. 3, pp. 361–373, Sep. 2017.
- [29] Z. Wang *et al.*, "Sample efficient actor-critic with experience replay," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2017, pp. 1–20.
- [30] Z. Wan, H. Li, H. He, and D. Prokhorov, "Model-free real-time EV charging scheduling based on deep reinforcement learning," *IEEE Trans. Smart Grid*, vol. 10, no. 5, pp. 5246–5257, Sep. 2019.
- [31] Y. Wu, E. Mansimov, R. B. Grosse, S. Liao, and J. Ba, "Scalable trust-region method for deep reinforcement learning using Kronecker-factored approximation," in *Proc. 31st Conf. Neural Inf. Process. Syst. (NIPS)*, 2017, pp. 1–10.
- [32] L. Yang and M. Wang, "Sample-optimal parametric Q-learning using linearly additive features," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 6995–7004.
- [33] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2016, pp. 1–14.
- [34] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017. [Online]. Available: <http://arxiv.org/abs/1707.06347>.
- [35] X. Tao, K. Ota, M. Dong, H. Qi, and K. Li, "Performance guaranteed computation offloading for mobile-edge cloud computing," *IEEE Wireless Commun. Lett.*, vol. 6, no. 6, pp. 774–777, Dec. 2017.
- [36] L. Yu, T. Jiang, and Y. Zou, "Distributed real-time energy management in data center microgrids," *IEEE Trans. Smart Grid*, vol. 9, no. 4, pp. 3748–3762, Jul. 2018.
- [37] Y. Xu, F. Pan, and L. Tong, "Dynamic scheduling for charging electric vehicles: A priority rule," *IEEE Trans. Autom. Control*, vol. 61, no. 12, pp. 4094–4099, Dec. 2016.

<sup>5</sup>If (48) holds for  $r > 1$  stages in  $\{t' + 1, \dots, t_j\}$ , then the inequality in (53) strictly holds when  $\gamma < 1$  and  $t_j - t' > r$ .

- [38] S. Senemar, A. R. Seifi, M. Rastegar, and M. Parvania, "Probabilistic optimal dynamic planning of onsite solar generation for residential energy hubs," *IEEE Syst. J.*, vol. 14, no. 1, pp. 832–841, Mar. 2020.
- [39] M. Moeini-Aghtaie, A. Abbaspour, and M. Fotuhi-Firuzabad, "Incorporating large-scale distant wind farms in probabilistic transmission expansion planning—Part I: Theory and algorithm," *IEEE Trans. Power Syst.*, vol. 27, no. 3, pp. 1585–1593, Aug. 2012.
- [40] J. Zhang *et al.*, "Joint resource allocation for latency-sensitive services over mobile edge computing networks with caching," *IEEE Internet Things J.*, vol. 6, no. 3, pp. 4283–4294, Jun. 2019.
- [41] Q. Fan and N. Ansari, "Application aware workload allocation for edge computing-based IoT," *IEEE Internet Things J.*, vol. 5, no. 3, pp. 2146–2153, Jun. 2018.
- [42] R. J. Williams, "Simple statistical gradient-following algorithms for connectionist reinforcement learning," *Mach. Learn.*, vol. 8, nos. 3–4, pp. 229–256, 1992.
- [43] K. Hornik, M. Stinchcombe, and H. White, "Multilayer feedforward networks are universal approximators," *Neural Netw.*, vol. 2, no. 5, pp. 359–366, 1989.
- [44] L. Xia, D. Miller, Z. Zhou, and N. Bambos, "Service rate control of tandem queues with power constraints," *IEEE Trans. Autom. Control*, vol. 62, no. 10, pp. 5111–5123, Oct. 2017.
- [45] A. Hill *et al.* (2018). *Stable Baselines*. [Online]. Available: <https://github.com/hill-a/stable-baselines>
- [46] F. Seide, L. Gang, and Y. Dong, "Conversational speech transcription using context-dependent deep neural networks," in *Proc. Int. Conf. Mach. Learn.*, 2012, pp. 1–2.

**Jiangliang Jin** received the B.S. degree from Fudan University, Shanghai, China, in 2010, the M.S. degree from Nanyang Technological University, Singapore, in 2013, and the Ph.D. degree from Singapore University of Technology and Design, Singapore, in 2018.

He is currently a Research Postdoctoral Fellow with the Chinese University of Hong Kong, Hong Kong. His research interest includes stochastic optimal control and demand side management for power systems.

**Yunjian Xu** (Member, IEEE) received the B.S. and M.S. degrees in electrical engineering from Tsinghua University, Beijing, China, in 2006 and 2008, respectively, and the Ph.D. degree from the Massachusetts Institute of Technology, Cambridge, MA, USA, in 2012.

He was an Postdoctoral Fellow with the Center for the Mathematics of Information, California Institute of Technology, Pasadena, CA, USA, from 2012 to 2013. Before joining the Chinese University of Hong Kong, Hong Kong, as an Assistant Professor, and he was an Assistant Professor with the Singapore University of Technology and Design, Singapore, from 2013 to 2017. His research interests lie in the intersection of stochastic optimal control and deep reinforcement learning, and its applications on power systems, Internet of Things, and electricity market.

Dr. Xu was a recipient of the MIT-Shell Energy Fellowship.